



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Oscillation-Free Discontinuous Galerkin Stabilization in MFEM

Master Thesis

Timo Zund

September 2026

Advisors: Dr. R. Käppeli

Seminar for Applied Mathematics,
Department of Mathematics, ETH Zürich

Abstract

[PLACEHOLDER ABSTRACT — Still needs:]

Principal findings and the final conclusion, after the numerical experiments have been agreed and evaluated. The text below is a provisional account of the problem and approach.

High-order discontinuous Galerkin methods provide local approximations of solutions to hyperbolic conservation laws, but unresolved discontinuities can produce spurious oscillations. This thesis investigates projection-based oscillation damping and its implementation on static curvilinear meshes. The approach combines an adapted oscillation-free discontinuous Galerkin (OFDG) filter with optional KXRCF troubled-cell indication, separating the strength of damping from the decision of where to apply it.

The formulation acts on finite element functions independently of their coefficient basis. Physical L^2 projections separate nonconstant components for damping while retaining cell averages. On curved elements, first physical derivatives are projected into the solution space and applied recursively to construct the derivative sensor. These operations represent successively projected derivatives rather than exact higher physical derivatives. The methodological account distinguishes published formulations from the adaptations used here and specifies the geometric and numerical assumptions needed for their realization in MFEM.

Contents

Contents	iii
1 Introduction	1
1.1 Motivation	1
1.2 Aim and research questions	1
1.3 Scope and approach	2
2 Mathematical background	3
2.1 Conservation laws and the quantity being conserved	3
2.2 What changes from continuous FEM?	3
2.2.1 Spaces, coefficients, and geometry	4
2.3 One element's weak form	4
2.3.1 Example: linear advection	5
2.4 Time stepping and the location of filtering	5
2.5 Nonlinear scalar equations and Euler	6
3 Methods	7
3.1 Baseline discretization and method composition	7
3.2 Adapted OFDG: damping without changing the cell mean	8
3.2.1 Relation to the published methods	8
3.2.2 Projections and a small example	8
3.2.3 Projection matrices in the solution basis	9
3.2.4 Which derivatives enter the sensor?	10
3.2.5 From reference derivatives to physical coefficients	11
3.2.6 Derivative jumps and their face evaluation	13
3.2.7 Damping rates and physical scales	15
3.2.8 Conservation, dissipation, and exact frozen decay	16
3.3 KXRCF troubled-cell indication	17
3.3.1 Inflow jumps and normalization	17
3.3.2 Conservative systems and transport direction	19
3.3.3 Physical geometry and numerical realization	19
3.3.4 Coupling to OFDG and limitations	20

3.4	Mathematical construction on curvilinear elements	20
3.4.1	What is represented, and what is differentiated?	20
3.4.2	Why the affine construction is insufficient	21
3.4.3	Physical mass matrices and nested projections	22
3.4.4	Projecting a physical derivative	23
3.4.5	Recursive higher derivatives: the precise approximation	25
3.4.6	Curved faces, normal directions, and length scales . .	26
3.5	The 2024 OEDG comparator	28
3.5.1	Scale-invariant interface sensor	28
3.5.2	Exact attenuation without a prescribed coefficient basis	29
3.5.3	Runge–Kutta time integration	30
3.6	Numerical controls	30
3.6.1	Euler admissibility	30
3.6.2	Experimental choices still to be fixed	31
3.7	Putting the OFDG building blocks together	31
3.8	Implementation assumptions and geometric scope	32
3.8.1	Reference data and physical operators	32
3.8.2	Distributed evaluation	33
3.8.3	Runtime data and storage costs	33
3.8.4	Verification criteria	34
4	Results	35
5	Discussion	37
6	Conclusion	39
A	Direct curved differentiation and surface geometry	41
A.1	Direct physical differentiation of the original DG function . .	42
A.1.1	The function is already defined in physical space . . .	42
A.1.2	First derivatives: why the Jacobian is still needed . . .	42
A.1.3	Second derivatives: differentiate the geometric factor too	43
A.1.4	A one-dimensional example through the third derivative	43
A.1.5	Higher derivatives from a local inverse Taylor series .	44
A.1.6	What direct differentiation improves, and what it cannot fix	46
A.2	The mathematics needed for the first-order surface example .	47
A.2.1	Two surface coordinates and three physical components	47
A.2.2	Deriving the tangential gradient	47
A.2.3	Surface area, edge length and the co-normal	48
A.2.4	The exact sphere map used in the prototype	49
A.2.5	Surface transport and the conservative DG form	49
A.2.6	Projected first derivatives and the order-one damping	50
A.3	Why higher surface derivatives are a separate extension . . .	51
A.3.1	Ambient and surface second derivatives answer different questions	51

A.3.2	A physical description of the surface Hessian	52
A.3.3	The same construction in reference coordinates	52
A.3.4	Why third order is not merely another scalar multi-index	53
Bibliography		55

Introduction

1.1 Motivation

Hyperbolic conservation laws describe the transport of quantities such as mass, momentum, and energy. Their solutions can contain both smooth structures and sharp fronts. Even smooth initial data can develop a discontinuity, so a numerical method must accommodate qualitatively different behaviour within the same calculation.

Discontinuous Galerkin (DG) methods combine finite element approximation within each mesh element with numerical fluxes between elements. Independent local polynomial representations allow high-order approximation without imposing continuity across interfaces [1]. Near an unresolved front, however, a polynomial approximation can develop overshoots and undershoots. Damping these oscillations indiscriminately can also remove smooth features that a high-order method is intended to resolve.

Projection-based damping offers a way to separate these concerns. The cell mean can be retained while nonconstant parts of the approximation are attenuated. The scalar OFDG formulation of Lu, Liu, and Shu [3] and its extension to hyperbolic systems [2] provide the starting point. The OEDG formulation of Peng, Sun, and Wu [4] further motivates normalized sensing and exact integration of a frozen damping step. A separate troubled-cell indicator can restrict the operation to selected cells. The central issue is then how to define the damping, its sensor, and its geometric realization consistently.

1.2 Aim and research questions

This thesis concerns a basis-independent realization of an adapted OFDG method in the finite element library MFEM. Here, basis independence

means that a change in coefficient representation does not change the filtered finite element function. The formulation combines physical projections, face-averaged derivative jumps, and optional KXRCF gating. It is an adaptation of the cited methods rather than a literal reproduction of a single paper.

The work is guided by three questions:

1. How can projection damping be expressed in the host finite element basis while preserving component-wise cell averages?
2. How should projection, derivative sensing, and geometric normalization be constructed when element maps are curved?
3. How do damping and selective activation affect accuracy and resolution of smooth and discontinuous solutions?

The first two questions determine the mathematical construction and its implementation. The third requires an agreed set of numerical experiments; its answer is deliberately left open in this draft.

1.3 Scope and approach

The construction uses a fixed mesh and a uniform solution order. It includes scalar conservation laws and conservative systems, with compressible Euler providing the system formulation. Static polynomial curved element maps are considered; moving meshes and variable solution order are outside the scope. On curved elements, differentiation generally leaves the mapped approximation space. The method therefore projects first physical derivatives back into that space and applies these projected operators recursively. The distinction between this construction and exact higher differentiation is central to the curvilinear extension.

The thesis separates three kinds of statements: identities derived from projection algebra, implementation choices defining the actual method, and empirical conclusions requiring numerical evidence. In particular, cell-mean preservation by the filter does not by itself establish positivity, full-scheme stability, or accuracy near a shock. Published analyses are used under their stated assumptions, not transferred automatically to the adapted method.

Chapter 2 introduces DG for a reader familiar with finite elements. Chapter 3 develops the stabilization methods, curvilinear extension, and their coefficient-space realization, followed by implementation assumptions, storage costs, and verification criteria. Results, Discussion, and Conclusion remain explicitly incomplete pending agreement on the numerical experiments. Exploratory results are retained in the separate preliminary brief.

Chapter 2

Mathematical background

This chapter assumes familiarity with the finite element idea: approximate a function in a finite-dimensional space and require a residual to vanish against test functions. DG retains that idea but removes continuity between elements. The price is that communication across each interface must be defined explicitly. We first explain this communication, then the time update that the filters modify. The introductory DG treatment follows the finite element viewpoint of [1].

2.1 Conservation laws and the quantity being conserved

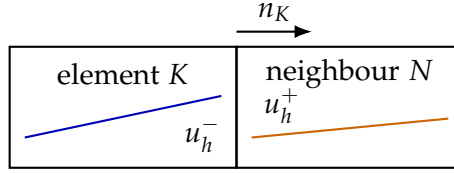
A conservation law describes a balance between local storage and transport:

$$\partial_t \mathbf{u} + \nabla \cdot \mathbf{f}(\mathbf{u}) = 0, \quad \mathbf{u}(x, 0) = \mathbf{u}_0(x). \quad (2.1)$$

Here $\mathbf{u} \in \mathbb{R}^m$ contains the conserved quantities, Ω is the spatial domain, and $\mathbf{f}$ gives their fluxes in the d coordinate directions. Boundary conditions supply the missing information on the exterior of Ω . Integrating over a region says that its total content changes only through flux across the region's boundary. We discuss the scalar case first; the same balance applies component by component to a system.

2.2 What changes from continuous FEM?

Let $\mathcal{T}_h$ be a mesh of elements K . In a continuous finite element space, neighbouring polynomial pieces must agree on a shared interface. In DG, each element has its own coefficients, so that agreement is not imposed. At a face, u_h^- denotes the value approached from inside the current element and u_h^+ the value approached from the neighbour. These are *traces*, not positive and negative parts of a number. Their difference is a *jump*.



independent traces; one shared numerical flux

Figure 2.1: A schematic pair of elements. The separate polynomial pieces may jump. A numerical flux gives both elements a consistent exchange across their common face. The drawn heights illustrate values, not element geometry.

This locality makes DG convenient for discontinuous solutions and distributed meshes. It does not by itself prevent oscillations: a high-order polynomial can overshoot near a shock even when its cell average is reasonable.

2.2.1 Spaces, coefficients, and geometry

Each physical element is the image of a reference element $\hat{K}$ under an invertible map T_K . Its order- k space is

$$V^k(K) = \{\hat{v} \circ T_K^{-1} : \hat{v} \in \hat{V}^k(\hat{K})\}, \quad V_h^k = \{v : v|_K \in V^k(K), K \in \mathcal{T}_h\}. \quad (2.2)$$

The reference family is total-degree P^k on a simplex, tensor-product Q^k on a quadrilateral or hexahedron, and the corresponding triangle-segment product on a prism. Thus “order k ” does not always mean total degree k . A curved map generally produces functions that are not polynomials in physical coordinates. All lower-order spaces in this work use the same map, so they remain nested: $V^0(K) \subset \dots \subset V^k(K)$.

With basis functions φ_i , the approximation is

$$u_h|_K = \sum_{i=1}^{N_K} U_{K,i} \varphi_i. \quad (2.3)$$

N_K is the number of local coefficients, or degrees of freedom (DOFs). Coefficients may be nodal values or modal expansion coefficients. The function represented by them, rather than the storage basis, is the object to which our projections and filters apply.

2.3 One element’s weak form

Multiply the scalar conservation law by a test function $w_h \in V^k(K)$, integrate over K , and integrate the spatial divergence by parts:

$$\int_K \partial_t u_h w_h dx - \int_K f(u_h) \cdot \nabla w_h dx + \int_{\partial K} \hat{f}(u_h^-, u_h^+, \mathbf{n}_K) w_h^- ds = 0. \quad (2.4)$$

2.4. Time stepping and the location of filtering

The first term is storage; the second accounts for variation inside the element; the third is exchange across its boundary. $\mathbf{n}_K$ is the outward unit normal. Since there are two traces, evaluating the physical flux from only one trace would leave the exchange ambiguous. The *numerical flux* $\hat{f}$ resolves that ambiguity.

The shared baseline uses the Rusanov flux

$$\hat{f}(u^-, u^+, \mathbf{n}) = \frac{1}{2}(f(u^-) + f(u^+)) \cdot \mathbf{n} - \frac{1}{2}\lambda(u^+ - u^-), \quad (2.5)$$

where λ bounds the relevant normal propagation speeds. The second term adds interface dissipation proportional to the jump. Equal traces recover the physical flux. Swapping traces and reversing the normal reverses the numerical flux, so an internal interface adds to one element exactly what it removes from the other, subject to consistent quadrature.

Choosing $w_h = 1$ gives the cell balance

$$\frac{d}{dt} \int_K u_h dx = - \int_{\partial K} \hat{f} ds. \quad (2.6)$$

Internal contributions cancel in the global sum. Domain totals therefore stay constant for periodic problems, but can change through open boundaries. A conservative filter must preserve the cell integral established by this DG update; it does not remove the boundary flux from this balance.

2.3.1 Example: linear advection

For $u_t + au_x = 0$ with constant $a > 0$, the physical flux is au . Setting $\lambda = |a|$ in Eq. (2.5) gives $\hat{f} = au^-$ at the right face: the upstream trace supplies the transported quantity. If two traces differ, the method communicates their effect through this flux rather than making the polynomials continuous. This example illustrates the general construction; it is not an additional governing assumption for OFDG.

2.4 Time stepping and the location of filtering

Substitution of the basis expansion into the weak form gives a finite system

$$M \frac{dU}{dt} = R(U), \quad M_{ij} = \int_K \varphi_i \varphi_j dx \quad \text{on each element.} \quad (2.7)$$

The mass matrix M is block-local; the residual R includes neighbour traces. An explicit Runge–Kutta method evaluates this residual at several intermediate *stages*. The implementation provides classical fourth-order RK (RK4), with a CFL step-size rule depending on mesh scale, order, and characteristic speed. CFL refers to the Courant–Friedrichs–Lewy restriction: the timestep

limits how far information travels relative to a cell width, with a stricter restriction at higher approximation order. A common CFL number therefore does not necessarily produce the same timestep for different evolving states. RK4's formal ODE order is not a proof of fourth-order accuracy for a nonlinear filtered shock calculation.

The terminology RKDG means DG in space combined with Runge–Kutta time stepping [3, 2]. In this work, adapted OFDG is normally applied after a complete step, whereas the OEDG comparator filters each RK stage. Their cadence is part of the comparison. Exact solution of the damping substep, introduced next, does not make the combined DG/filter evolution exact.

2.5 Nonlinear scalar equations and Euler

Burgers' equation uses $f(u) = u^2/2$, so its propagation speed depends on the state; initially smooth data can develop a shock. Compressible Euler has $m = d + 2$ conserved components,

$$\mathbf{u} = (\rho, \rho \mathbf{v}, E)^\top, \quad p = (\gamma - 1) \left(E - \frac{1}{2} \rho |\mathbf{v}|^2 \right). \quad (2.8)$$

Here ρ is density, $\mathbf{v}$ velocity, E total energy per unit volume, p pressure, and $\gamma = 1.4$ the ratio of specific heats used by the Euler implementation. The flux in coordinate direction j is

$$\mathbf{f}_j = (\rho v_j, \rho v_j \mathbf{v} + p \mathbf{e}_j, (E + p) v_j)^\top, \quad (2.9)$$

with $\mathbf{e}_j$ the coordinate unit vector. The maximum normal wave speed is $|\mathbf{v} \cdot \mathbf{n}| + c$, where $c = \sqrt{\gamma p / \rho}$ is the sound speed. This differs from the signed fluid transport velocity used by the KXRCF inflow decision. Density and pressure must remain positive for these states and speeds to be physically admissible.

Chapter 3

Methods

This chapter defines the numerical method used in this work. The DG formulation in Chapter 2 supplies the conservative evolution. Adapted OFDG adds local damping, and KXRCF optionally determines which cells receive it. The curvilinear construction gives these operations their physical meaning on mapped elements. OEDG is described as a related method available for comparison, followed by numerical controls and implementation assumptions.

3.1 Baseline discretization and method composition

The baseline is the weak DG formulation Eq. (2.4) with the Rusanov flux Eq. (2.5). It uses a fixed scalar DG space, repeated for each conservative component, and explicit Runge–Kutta time integration. Periodic boundaries identify the corresponding traces; physical boundaries supply an exterior state through the DG flux. The oscillation sensor uses interior interfaces and does not replace this boundary treatment.

As a roadmap, denote a complete RKDG step by $\mathcal{H}_{\Delta t}$ and the subsequent damping operation by $\mathcal{F}_{\Delta t}$, where Δt is the physical timestep. The vector U^n contains the DG coefficients at time t_n and U^* is the candidate state before damping. The two operations are composed as

$$U^* = \mathcal{H}_{\Delta t}(U^n), \quad U^{n+1} = \mathcal{F}_{\Delta t}(U^*), \quad (3.1)$$

The construction of $\mathcal{F}$ is developed from projections (Section 3.2.2), derivative jumps (Section 3.2.4), and damping rates (Section 3.2.7), before giving the assembled operator and its exact frozen decay in Section 3.2.8. With KXRCF, the active set is computed from U^* before applying $\mathcal{F}$. Without an oscillation filter, $\mathcal{F}$ is the identity. The implementation defaults to post-step OFDG and stage-wise OEDG; the location of filtering is part of the numer-

ical method, not an incidental software setting. Euler additionally uses the admissibility control in Section 3.6.

3.2 Adapted OFDG: damping without changing the cell mean

The DG state from Chapter 2 may contain unresolved oscillations. Adapted OFDG reduces nonconstant content within each element while preserving its mean. Three ingredients determine the operation: nested projections separate orders of variation, interelement jumps measure roughness, and wave-speed-scaled rates determine how strongly each part is damped.

3.2.1 Relation to the published methods

The name *adapted OFDG* is used throughout this report for the specific hybrid construction implemented here. Projection damping originates in [3, 2]; global scale normalization and exponential decay are motivated by [4]. The earlier multidimensional OFDG sensors evaluate vertex jumps, and the system formulation uses characteristic variables. Here we average absolute jumps over faces and form separate rates for physical conservative components. KXRCF gating is optional.

These choices define an implementation variant, not a literal reproduction of one paper. The conservation and damping identities below follow from projection algebra. Published convergence theorems for different sensors are not a proof for this variant, particularly on curved elements with projected derivatives.

3.2.2 Projections and a small example

For the nested mapped spaces from Eq. (2.2), let Π_K^r be the physical L^2 projection into $V^r(K)$:

$$(\Pi_K^r w - w, v)_K = 0 \quad \text{for every } v \in V^r(K), \quad (p, q)_K = \int_K pq \, dx. \quad (3.2)$$

This is the closest approximation in the integral squared-error norm. Because constants belong to every space, the projection preserves the element integral. In particular $\Pi_K^0 w = |K|^{-1} \int_K w \, dx$ is the constant cell mean.

For example, on $[-1, 1]$ write

$$w(x) = a_0 + a_1 x + a_2(x^2 - 1/3). \quad (3.3)$$

The last two terms have zero mean. Reducing their coefficients changes the slope and curvature but leaves a_0 unchanged. This orthogonal example is

an explanation, not a required storage basis: the code assembles projections in MFEM's existing scalar finite element basis.

Figure 3.1 illustrates how attenuating zero-mean terms changes a profile while preserving its average.

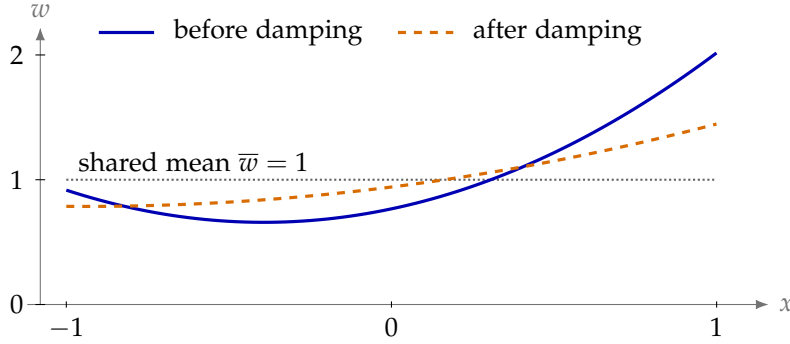


Figure 3.1: An analytic illustration of projection damping on $[-1, 1]$. The solid curve is $w = 1 + 0.55x + 0.7(x^2 - 1/3)$. Multiplying its linear and quadratic shells by 0.6 and 0.25 gives the dashed curve. Both nonconstant basis functions have zero integral, so the cell mean stays at one even though the profile changes. The factors are chosen for illustration; these are not simulation results or a positivity guarantee.

Define the remainders and shells

$$\mathcal{R}_K^{(l)} = I - \Pi_K^{l-1}, \quad \Pi_K^{-1} := \Pi_K^0, \quad (3.4)$$

$$\mathcal{Q}_K^{(0)} = \Pi_K^0, \quad \mathcal{Q}_K^{(j)} = \Pi_K^j - \Pi_K^{j-1}, \quad 1 \leq j \leq k. \quad (3.5)$$

Here I is the identity operator. In particular, $\mathcal{R}_K^{(0)} = \mathcal{R}_K^{(1)} = I - \Pi_K^0$: both remove the constant mean and retain everything else. For $l \geq 2$, the remainder removes the whole order- $(l-1)$ subspace. The special convention for Π_K^{-1} ensures that even the solution-jump contribution cannot damp the mean. A shell contains the variation added when increasing the reference-space order from $j-1$ to j . These shells are mutually orthogonal in the physical inner product. On tensor-product or curved elements, “shell j ” means this nested space difference; it does not mean total physical polynomial degree j .

3.2.3 Projection matrices in the solution basis

To apply these function-space operations to the stored DG coefficients, we need matrices in the existing solution basis. Write $u_h = \sum_{i=1}^{N_k} U_i \varphi_i$ on K , so $U \in \mathbb{R}^{N_k}$ is a coefficient vector, not a list of point values in general. The physical inner product in Eq. (3.2) supplies the matrix entries below; its evaluation on curved elements is derived in Eq. (3.43).

3. METHODS

Let $\{\varphi_i\}_{i=1}^{N_k}$ be the local degree- k basis and $\{\psi_a^{(r)}\}_{a=1}^{N_r}$ the degree- r basis. The code assembles

$$(M_k)_{ij} = (\varphi_i, \varphi_j)_K, \quad (M_r)_{ab} = (\psi_a^{(r)}, \psi_b^{(r)})_K, \quad (3.6)$$

$$(B_r)_{ai} = (\psi_a^{(r)}, \varphi_i)_K. \quad (3.7)$$

Here $M_k \in \mathbb{R}^{N_k \times N_k}$ and $M_r \in \mathbb{R}^{N_r \times N_r}$ are mass matrices, while $B_r \in \mathbb{R}^{N_r \times N_k}$ transfers the inner products with u_h to the lower-order test basis. Testing the projection equation gives $M_r C_r = B_r U$. The lower-order coefficients C_r represent $\Pi_K^r u_h$; lifting this same function to the original basis gives $M_k U_r = B_r^\top C_r$. Eliminating C_r yields the projector, and subtraction from U yields the remainder. The matrix representation of $I - \Pi_K^r$ in the degree- k coefficient basis is

$$Q_r = M_k - B_r^\top M_r^{-1} B_r, \quad S_r = M_k^{-1} Q_r = I - M_k^{-1} B_r^\top M_r^{-1} B_r. \quad (3.8)$$

Thus applying S_r to a degree- k coefficient vector directly returns the coefficients of the remainder $w - \Pi_K^r w$. Here I is the $N_k \times N_k$ identity matrix, and inverse notation denotes linear solves. The code stores S_r for $0 \leq r \leq k$.

Crucially, this construction never converts the solution to Legendre or other orthogonal modal coefficients. It represents the same projection operator in whatever supported scalar polynomial DG basis the host framework supplies. Consequently, a nodal basis, a modal basis, or another polynomial coefficient basis produces the same function-space operation up to quadrature and roundoff. This basis-agnostic realization is what makes the implementation portable: a second framework need not add a special OFDG solution basis, provided it can form the corresponding polynomial-space projections.

On an affine element, all three mass and coupling matrices contain the same constant physical Jacobian factor, which cancels in S_r . Therefore affine projection matrices are reused per finite-element signature. For curved elements every mass and coupling matrix includes the physical Jacobian weight at each quadrature point. The lower-order spaces are mapped reference polynomial spaces; their physical L^2 projectors remain nested.

For example, $S_0 U$ is the nonconstant remainder, and $(S_{j-1} - S_j)U$ represents shell j . Since $S_k = 0$, these stored remainders suffice for every shell in Eq. (3.5).

3.2.4 Which derivatives enter the sensor?

First distinguish an exact derivative from the projected operation used in its numerical approximation. A *multi-index* is a tuple of nonnegative integers. For spatial dimension d , define

$$\alpha = (\alpha_1, \dots, \alpha_d) \in \mathbb{N}_0^d, \quad |\alpha| = \sum_{j=1}^d \alpha_j, \quad \partial^\alpha = \partial_{x_1}^{\alpha_1} \dots \partial_{x_d}^{\alpha_d}. \quad (3.9)$$

Here $\mathbb{N}_0 = \{0, 1, 2, \dots\}$, α_j counts differentiations in coordinate x_j , and $|\alpha|$ is their total order. For example, in two dimensions $(1, 0)$ means ∂_x , $(0, 1)$ means ∂_y , $(1, 1)$ means a mixed second derivative, and $(2, 1)$ means two x -derivatives and one y -derivative. The zero multi-index means no differentiation: $\partial^0 v = v$. For sufficiently smooth functions within an element, exact mixed partial derivatives commute, including on curved elements. This statement concerns the physical function, not its discrete approximation.

Define the projected first-derivative operator by

$$\mathcal{D}_{K,j} v = \Pi_K^k(\partial_{x_j} v). \quad (3.10)$$

It first differentiates in physical coordinate x_j , then projects the result back into $V^k(K)$. For repeated application, this report uses the explicit convention

$$\mathcal{D}_K^\alpha = \mathcal{D}_{K,d}^{\alpha_d} \cdots \mathcal{D}_{K,2}^{\alpha_2} \mathcal{D}_{K,1}^{\alpha_1}, \quad \mathcal{D}_K^0 = I. \quad (3.11)$$

Operators act from right to left: all x_1 operations come first, then x_2 , and so on. This is the sequence selected by the implementation. Thus $\mathcal{D}_K^{(1,1)} v = \mathcal{D}_{K,y} \mathcal{D}_{K,x} v$ in 2D; the superscript labels that particular sequence, not all its permutations.

On affine elements whose reference family is closed under differentiation, these operators recover the exact polynomial derivatives up to numerical precision, and their order is immaterial. On a curved element they represent *successively projected* physical derivatives. Inserting a projection between differentiations can destroy commutation. Consequently $\mathcal{D}_{K,y} \mathcal{D}_{K,x} v$ and $\mathcal{D}_{K,x} \mathcal{D}_{K,y} v$ need not agree, even though $\partial_y \partial_x v = \partial_x \partial_y v$. Section 3.4.5 derives the difference and explains what the multi-index storage does and does not imply.

3.2.5 From reference derivatives to physical coefficients

The sensor needs derivative traces on both sides of a face. We first construct derivative coefficients within each element, then evaluate their traces. This avoids requesting higher basis derivatives from MFEM. Let $\widehat{\varphi}_i = \varphi_i \circ T_K$ be the reference basis and let $\{\boldsymbol{\xi}_q\}_{q=1}^{N_k}$ be the finite element's interpolation points. These are distinct from the integration points used for volume or face quadrature. The supported element must provide an invertible evaluation matrix at these points. For each reference direction $r \in \{1, \dots, d\}$, define the two $N_k \times N_k$ matrices

$$V_{qi} = \widehat{\varphi}_i(\boldsymbol{\xi}_q), \quad G_{qi}^{(r)} = \frac{\partial \widehat{\varphi}_i}{\partial \xi_r}(\boldsymbol{\xi}_q). \quad (3.12)$$

Rows index sample points and columns index basis functions. Thus $V\mathbf{u}$ gives solution values at the interpolation points, while $G^{(r)}\mathbf{u}$ contains

3. METHODS

reference-derivative values at the interpolation points. To express those derivative values as coefficients, solve $V\mathbf{u}_{\xi_r} = G^{(r)}\mathbf{u}$. Because reference polynomial differentiation stays in the supported space, this reconstructs the derivative exactly up to roundoff. Therefore

$$\mathbf{u}_{\xi_r} = D_{\xi_r}\mathbf{u}, \quad D_{\xi_r} = V^{-1}G^{(r)}. \quad (3.13)$$

The code's `AssembleReferenceDerivatives` builds V with MFEM's `CalcShape`, builds $G^{(r)}$ with `CalcDShape`, and explicitly forms $V^{-1}G^{(r)}$. For a Lagrange nodal basis at its own nodes, $V = I$. Keeping V explicit also handles supported bases whose coefficients are not those nodal values; invertibility is essential.

For a small example, use $(1, \xi, \xi^2)$ on $[0, 1]$ and sample at $0, \frac{1}{2}, 1$. Then

$$V = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1/2 & 1/4 \\ 1 & 1 & 1 \end{pmatrix}, \quad G = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 2 \end{pmatrix}, \quad D_{\xi} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 2 \\ 0 & 0 & 0 \end{pmatrix}. \quad (3.14)$$

For $\mathbf{u} = (a, b, c)^T$, $G\mathbf{u}$ contains the sampled slopes $(b, b+c, b+2c)^T$, whereas $D_{\xi}\mathbf{u} = (b, 2c, 0)^T$ contains the coefficients of $b+2c\xi$. This is why values and coefficients cannot generally be interchanged.

Now introduce the affine element map

$$\mathbf{x} = T_K(\boldsymbol{\xi}) = \mathbf{b}_K + J_K\boldsymbol{\xi}, \quad (J_K)_{jr} = \frac{\partial x_j}{\partial \xi_r}. \quad (3.15)$$

The vector $\mathbf{b}_K$ translates the element and the invertible $d \times d$ matrix J_K describes scaling, rotation, and shear. It is constant within this element but may differ between elements. The chain rule $\partial_{x_j} = \sum_r (J_K^{-1})_{rj} \partial_{\xi_r}$ gives the physical coefficient matrices

$$D_{x_j}^K = \sum_{r=1}^d (J_K^{-1})_{rj} D_{\xi_r}. \quad (3.16)$$

The inverse Jacobian J_K^{-1} is read once at the reference-element centre and one set of matrices $\{D_{x_j}^K\}_{j=1}^d$ is stored for every element.

For a curved map J_K varies within the element, so the constant combination in Eq. (3.16) is insufficient. The matrix representing Eq. (3.10) is instead $D_{K,j} = (M_K^K)^{-1}G_{K,j}$, with the physical integral matrix $G_{K,j}$ derived in Eq. (3.50). Unlike $G^{(r)}$, which stores pointwise reference derivative values, $G_{K,j}$ stores physical inner products. Repeated application follows Eq. (3.11); the parent rule and the projection error are developed in Section 3.4.5.

Coordinate derivatives and normal derivatives. A physical coordinate derivative differentiates along x_j . At a face point x_q with outward unit normal $\mathbf{n}_K(x_q) = (n_1(x_q), \dots, n_d(x_q))$, the normal derivative is

$$\partial_{n_K} u_h(x_q) = \sum_{j=1}^d n_j(x_q) \partial_{x_j} u_h(x_q). \quad (3.17)$$

On an affine planar face it can be evaluated from $\sum_j n_j D_{x_j}^K \mathbf{u}$ and the face basis values. On a curved face, contracting the projected coordinate derivatives gives an approximation $\sum_j n_j(x_q) (\mathcal{D}_{K,j} u_h)(x_q)$; the normal must be evaluated at that point. This need not equal the exact normal derivative or its single physical projection. The implemented OFDG sensor uses jumps of the *coordinate derivatives* in Eq. (3.19), not jumps of this contraction. Normals instead enter its wave speeds and KXRCF's inflow selection. No separate normal-derivative matrix is required by these sensors.

Only first reference derivatives of the supplied finite-element basis are requested from MFEM. Repeated application of the resulting coefficient operators generates every higher derivative needed by OFDG. In framework terms, the construction requires scalar polynomial DG basis values and first derivatives, quadrature, nested lower-order polynomial spaces or equivalent projection operators, element and face transformations, and local degree-of-freedom access. It therefore avoids a modal-basis requirement, but it does not claim support for arbitrary nonpolynomial, $H(\text{div})$, $H(\text{curl})$, or mixed finite-element spaces.

All multi-indices with total degree at most k are organized in a directed acyclic graph. Each nonzero multi-index selects one parent of degree one less and one physical derivative direction. Walking this graph evaluates every required derivative once. The number of stored derivative states per element and component is

$$\sum_{l=0}^k \binom{l+d-1}{d-1} = \binom{k+d}{d}. \quad (3.18)$$

3.2.6 Derivative jumps and their face evaluation

For an interior face $F = \partial K^- \cap \partial K^+$, let w^- and w^+ be traces from its two adjacent elements. The jump is their difference. For component $c \in \{1, \dots, m\}$ and derivative order $l \in \{0, \dots, k\}$, the sensor adds the mean absolute jumps of all projected derivatives of total order l . With $|F|$ the physical face measure, these definitions read

$$[[w]]_F = w^-|_F - w^+|_F, \quad J_{F,c}^{(l)} = \sum_{|\alpha|=l} \frac{1}{|F|} \int_F |[[\mathcal{D}^\alpha u_{h,c}]]_F| \, ds. \quad (3.19)$$

3. METHODS

The sum runs over all nonnegative multi-indices with $|\alpha| = l$, once each. In 2D, the $l = 1$ sum includes $(1, 0)$ and $(0, 1)$; the $l = 2$ sum includes $(2, 0)$, $(1, 1)$, and $(0, 2)$. The mixed term is counted once, using Eq. (3.11); there is no additional reversed-order term or multiplicity factor. For $l = 0$ the only index is $(0, 0)$, so the sensor measures jumps of the solution itself. Each trace uses its own element's derivative operator at the same physical face point. Face measure $|F|$ is length in 2D or area in 3D; a 1D point face uses the ordinary absolute jump. Taking the absolute value before integration prevents cancellation. For constant traces 2 and 1.25, $J_F^{(0)} = 0.75$ regardless of face size.

MFEM remains the sole source of mesh connectivity. The setup loops over MFEM face numbers and obtains the two adjacent elements from MFEM's face-transformation data. Boundary faces are skipped. For each interior affine face, a quadrature rule of order $2k + 1$ is selected. Curved faces use an overintegrated rule that also depends on the geometry order. Let x_q be a physical face quadrature point and let $\xi_{F,q}^\pm = T_{K^\pm}^{-1}(x_q)$ be its reference coordinates in the two adjacent elements. If their coefficient counts are N_k^- and N_k^+ , then $E_F^\pm$ has size $N_{q,F} \times N_k^\pm$. Each row converts one side's coefficients to a trace value at x_q . The following data are cached directly at the MFEM face index:

$$(E_F^-)_{qi} = \hat{\varphi}_i(\xi_{F,q}^-), \quad (E_F^+)_{qi} = \hat{\varphi}_i(\xi_{F,q}^+), \quad \hat{\omega}_q = \frac{\omega_q J_F(\xi_q)}{\sum_p \omega_p J_F(\xi_p)}. \quad (3.20)$$

Here J_F is the physical face Jacobian. For affine faces it is constant and cancels between the integral and $|F|$ in (3.19); hence the normalized reference weights $\hat{\omega}_q$ are sufficient. All derivative traces and all components on one side of a face are evaluated together in one dense matrix multiplication. There is no face-pattern matching or deduplication.

If $\mathbf{u}_\alpha^-$ and $\mathbf{u}_\alpha^+$ are the coefficient vectors of the same physical derivative on the two neighboring elements, the actual quadrature calculation is

$$J_{F,c}^{(l)} \approx \sum_{|\alpha|=l} \sum_{q=1}^{N_{q,F}} \hat{\omega}_q \left| (E_F^- \mathbf{u}_\alpha^-)_q - (E_F^+ \mathbf{u}_\alpha^+)_q \right|. \quad (3.21)$$

The implementation therefore performs one subtraction and one absolute value for each derivative, component, and quadrature point. No squared-jump accumulator or final square root is required.

Thus the full coefficient-to-sensor route is: differentiate the element coefficients, multiply by E_F^- and E_F^+ , subtract matching traces, take absolute values, and average with normalized physical face weights. For curved elements, the pointwise normal and surface factor are derived in Eqs. (3.57) and (3.58).

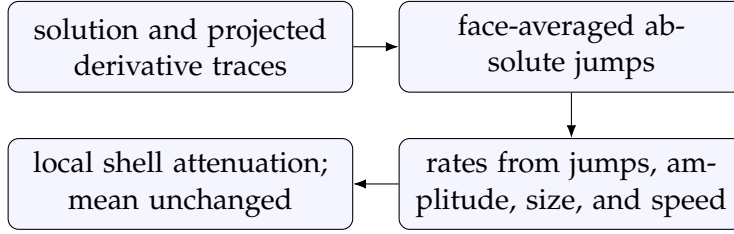


Figure 3.2: Adapted OFDG separates face-based sensing from element-local damping. KXRCF, when enabled, selects which elements receive the final operation.

3.2.7 Damping rates and physical scales

Let $\bar{u}_c = |\Omega|^{-1} \int_{\Omega} u_{h,c} dx$ and define the ideal component amplitude

$$A_c = \|u_{h,c} - \bar{u}_c\|_{L^\infty(\Omega)}. \quad (3.22)$$

The code approximates this amplitude from volume-quadrature samples and an integrated mean; it does not optimize the continuous polynomial maximum. Adapted OFDG sets its component rates to zero when the sampled amplitude is at most 10^{-14} . This absolute cutoff limits exact rescaling invariance near vanishing amplitudes.

The one-sided speed $\beta_{K,F}$ is the largest sampled normal radius on face F using the state on side K . Prescribed advection uses $|\mathbf{a} \cdot \mathbf{n}_K|$; Euler uses $|\mathbf{v}_K \cdot \mathbf{n}_K| + c_K$ evaluated from face traces. The other element can have a different radius. Normals are evaluated at the same physical face quadrature points. Unit speed is a compatibility policy, rather than the state-dependent physical-speed policy defined here.

With $k \geq 1$ the solution-space order, $|K|$ and $|\hat{K}|$ the physical and reference element volumes, and $h_K = (|K|/|\hat{K}|)^{1/d}$, the rates are

$$\delta_{K,c}^{(l)} = \frac{2l+1}{2(2k-1)l!} \frac{h_K^{l-1}}{A_c} \sum_{\substack{F \subset \partial K \\ F \in \mathcal{F}_h^i}} \beta_{K,F} J_{F,c'}^{(l)} \quad 0 \leq l \leq k, \quad (3.23)$$

Here $\mathcal{F}_h^i$ denotes the set of interior mesh faces, ∂K is the element boundary, and $l!$ is the factorial of the total derivative order, with $0! = 1$. The sum collects contributions from interior faces of K only. Physical boundary conditions belong to the DG flux operator. Face and volume integrals in the implementation use quadrature, including geometry-dependent weights on curved cells.

The rate has inverse-time units: the derivative jump contributes u/length^l , h_K^{l-1} contributes length^{l-1} , speed contributes $\text{length}/\text{time}$, and A_c divides out the state scale. At fixed geometry and amplitude, larger speeds or jumps

strengthen attenuation; the factor h_K^{l-1} sets the length scaling for each derivative order. For $k = 2$, $l = 0$, $h_K = 0.1$, $A = 1$, $\beta = 2$, and one jump $J = 0.3$, the contribution is $(1/6)(0.1^{-1})(2)(0.3) = 1$ per unit time.

With fixed prescribed speeds and away from the cutoff, the jump/amplitude ratio is unchanged under $u_c \mapsto \lambda u_c + b$, $\lambda \neq 0$. Arbitrary component scaling or shifting of Euler or Burgers states can change the physical speeds, so the complete nonlinear filtering map does not inherit that statement unconditionally.

3.2.8 Conservation, dissipation, and exact frozen decay

We now combine the two element-local building blocks. For each derivative order l , the rate $\delta_{K,c}^{(l)}$ measures how strongly to act, and the remainder $\mathcal{R}_K^{(l)} u_{h,c}$ selects which content may be removed. Adding these contributions defines the damping operator

$$\mathcal{D}_{K,c} u_{h,c} = \sum_{l=0}^k \delta_{K,c}^{(l)} \mathcal{R}_K^{(l)} u_{h,c}. \quad (3.24)$$

Every remainder has zero integral. For ideal physical projections,

$$\int_K \mathcal{D}_{K,c} u_{h,c} dx = 0, \quad (3.25)$$

$$(\mathcal{D}_{K,c} u_{h,c}, u_{h,c})_K = \sum_{l=0}^k \delta_{K,c}^{(l)} \|\mathcal{R}_K^{(l)} u_{h,c}\|_{L^2(K)}^2 \geq 0. \quad (3.26)$$

Thus subtracting $\mathcal{D}u$ removes nonconstant squared-norm content without changing the cell mean. These are damping-step identities, not a positivity or entropy-stability proof for the full nonlinear time update. The numerical realization preserves its quadrature-weighted physical cell means to round-off; agreement with exact integrals also depends on quadrature.

Freeze the rates at the candidate state and solve $\partial_s w + \mathcal{D}w = 0$ for a pseudo-time interval τ . Nested projections give the explicit solution

$$w(\tau) = \Pi_K^0 u_{h,c} + \sum_{j=1}^k \exp\left(-\tau \sum_{l=0}^j \delta_{K,c}^{(l)}\right) (\Pi_K^j - \Pi_K^{j-1}) u_{h,c}. \quad (3.27)$$

The mean is copied and each higher shell is multiplied by a factor between zero and one. For order three, rates $l = 0$ and $l = 1$ affect all nonconstant shells; rate $l = 2$ affects shells two and three; rate $l = 3$ affects shell three. “Exact” refers to this frozen damping equation. The rates are recomputed from the evolving solution, and operator splitting remains part of the method.

The public stabilization routine returns coefficients of $\mathcal{D}u$ for a caller that wants a semidiscrete term. The default post-step implementation uses the exponential CompDecay map after a full RK4 step, with τ equal to the step size. Basis-independent matrix construction is given in Section 3.2.3.

3.3 KXRCF troubled-cell indication

A troubled-cell indicator decides where a separate stabilization should act; it does not itself change the solution. The KXRCF indicator, named after Krivodonova, Xin, Remacle, Chevaugeon, and Flaherty, supplies that decision in this implementation. Adapted OFDG then damps the selected elements using the operator in Section 3.2.

[PLACEHOLDER KX-SOURCE — Still needs:]

Review and approve the original KXRCF paper, which is not currently in the papers folder. The formulas below define the implemented indicator; historical attribution and comparison with the original analysis still require that source. Its bibliographic entry is in suggested-references.

3.3.1 Inflow jumps and normalization

Let K have outward unit normal $\mathbf{n}_K$ and neighbour N . Define $[u_h]_{K,N} = u_h|_K - u_h|_N$. The part of the interior boundary on which transport enters K is identified by the signed normal transport velocity v_n . For a prescribed, single-valued advection velocity $\mathbf{a}$, this is $v_n = \mathbf{a} \cdot \mathbf{n}_K$: negative values mean inflow, positive values mean outflow. The choices for two-sided velocities and nonlinear systems are specified in Section 3.3.2. Define

$$\partial K^- = \{x \in \partial K \text{ on interior faces} : v_n(x) < 0\}. \quad (3.28)$$

Figure 3.3 shows which face enters this integral in a simple rightward-flow example.

The indicator combines a signed jump integral with three normalization factors. The physical measure $|\partial K^-|$ turns that integral into an inflow average. The radius r_K measures element size: it is half the cell width in 1D and the largest sampled distance from the mapped centre to the boundary in higher dimensions. The solution scale is $S_K = |\bar{u}_K|$ in 1D, with $\bar{u}_K = |K|^{-1} \int_K u_h dx$; in higher dimensions S_K is the largest sampled $|u_h|$ within K . Precise geometric sampling and the small-scale safeguard are given in Section 3.3.3.

Let k be the reference solution order, τ the detector threshold, and $\mathcal{A}$ the set of elements selected for damping. The assembled indicator and selection

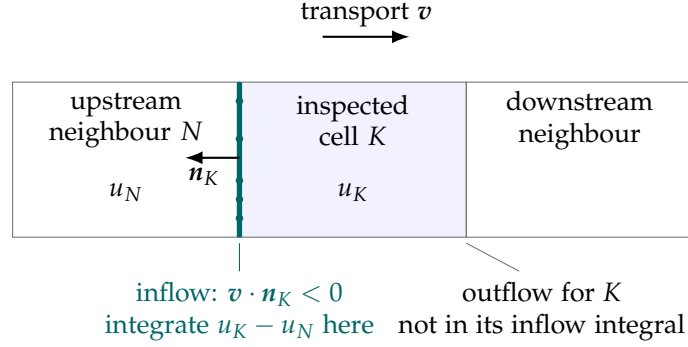


Figure 3.3: KXRCF on an interior patch with uniform rightward transport. For the inspected cell K , the left face is inflow, the right face is outflow, and the horizontal faces are tangential. The highlighted face enters K 's signed jump integral. Colours identify the inspected cell and face, not a computed troubled-cell classification. For curved or varying flow, the sign is tested separately at each face quadrature point.

rule are

$$I_K = \frac{\left| \int_{\partial K^-} [u_h]_{K,N} ds \right|}{r_K^{(k+1)/2} |\partial K^-| S_K}, \quad K \in \mathcal{A} \iff I_K > \tau. \quad (3.29)$$

The default threshold is $\tau = 1$; $I_K = 0$ when there is no inflow measure. The radius is denoted by r_K here to distinguish it from the OFDG length h_K . The face measure averages the jump, and S_K normalizes the state magnitude. The absolute value is taken *after* integration, so signed contributions can cancel. This differs from the absolute-jump integral used by the OFDG sensor.

For motivation, suppose the averaged signed jump is $\mathcal{O}(r_K^{k+1})$ in a smooth region and $S_K = \mathcal{O}(1)$. Then $I_K = \mathcal{O}(r_K^{(k+1)/2})$ tends to zero under refinement. An unresolved discontinuity with a noncancelling $\mathcal{O}(1)$ averaged jump instead gives $I_K = \mathcal{O}(r_K^{-(k+1)/2})$. These conditional estimates explain the normalization; they are not a convergence theorem or a guarantee that every discontinuity is detected. Cancellation and a small local scale can both affect the decision. The numerical value also depends on the coordinate scaling through r_K , so a threshold must be interpreted with that scaling specified.

In one dimension, with positive velocity and cell width Δx_j , the inflow boundary is the left endpoint and $r_K = \Delta x_j/2$. The implemented scale is $S_K = |\bar{u}_K|$, giving

$$I_j = \frac{|u_h(x_{j-1/2}^+) - u_h(x_{j-1/2}^-)|}{(\Delta x_j/2)^{(k+1)/2} |\bar{u}_j|}, \quad (3.30)$$

when the denominator is resolved. The relative scale floor in Section 3.3.3 handles a vanishing or very small mean.

3.3.2 Conservative systems and transport direction

For each conservative component c , the implementation forms $I_{K,c}$ from Eq. (3.29) and pools the decision:

$$I_K = \max_c I_{K,c}, \quad \mathcal{A} = \{K : I_K > \tau\}. \quad (3.31)$$

All components are therefore filtered on the same active set. This pooling, together with the following transport choices, defines the project-specific system extension; it should not be mistaken for characteristic-variable limiting.

For prescribed advection, v_n is the arithmetic mean of the normal velocities on the two sides. For one-dimensional Burgers flow it is $v_n = \frac{1}{2}(u_1 + u_2)n$. For Euler it is

$$v_n = \frac{1}{2}(v_1 + v_2) \cdot \mathbf{n}. \quad (3.32)$$

This is a fluid transport direction, not the Euler spectral radius. Characteristic fields can travel in both directions at a subsonic interface; a single signed velocity does not describe them all. OFDG uses the full one-sided radii $|\mathbf{v}_i \cdot \mathbf{n}| + c_i$ for damping strength, whereas the sign of Eq. (3.32) controls KXRCF inflow only.

3.3.3 Physical geometry and numerical realization

In higher dimensions, r_K is the largest sampled distance from the mapped reference centre to mapped vertices and, on curved elements, boundary quadrature points. It is a sampled geometric scale rather than an exact circumradius for an arbitrary curved cell. Face quadrature supplies the signed jump integral and the physical inflow measure. Unit normals and the transport sign are evaluated at those points. Transport within 128ε times the local speed scale is treated as tangential, where ε is machine precision.

For component c , the scale used in the denominator is

$$S_{K,c}^{\text{eff}} = \max(S_{K,c}, 10^{-14} S_c^{\text{max}}), \quad S_c^{\text{max}} = \max_K S_{K,c}. \quad (3.33)$$

Globally zero components are skipped. This avoids division by a vanishing local scale while preserving the multiplicative scale dependence away from numerical roundoff. Physical boundary faces are omitted because the indicator uses neighbouring DG traces; boundary conditions remain part of the DG evolution.

Element and face evaluation matrices, geometric weights, normals, and the factor $r_K^{(k+1)/2}$ are prepared at construction. Local face transformations are retained, but solution values and transport directions are reevaluated on every application. At shared MPI faces, neighbour coefficients are imported and each rank accumulates only its own element's contribution. Geometry and solution spaces must remain fixed during the lifetime of the indicator.

3.3.4 Coupling to OFDG and limitations

KXRCF determines $\mathcal{A}$ from the candidate DG state. An interior face is needed by the OFDG sensor whenever at least one adjacent element is active; this ensures that active elements receive all their interface contributions. The damping map is applied only to active elements. Its mean-preservation identity therefore survives gating, and inactive coefficients remain unchanged.

KXRCF requires no solution derivatives, although the subsequent OFDG sensor does. A small active set does not eliminate the cost of cell normalization, inflow integration, or OFDG derivative preparation. Nor does the indicator alone guarantee nonoscillatory or positive solutions. Threshold sensitivity, missed fronts, and unnecessary activation in smooth regions require empirical assessment in the eventual numerical study.

3.4 Mathematical construction on curvilinear elements

The affine implementation relies on two simplifications: the Jacobian is constant within an element, and differentiation stays inside the represented polynomial space. A curved map removes both simplifications. Our extension keeps the physical projection idea of OFDG, but assembles its operators for each curved element and projects each derivative back into the solution space. This section extends the coefficient construction in Sections 3.2.3, 3.2.5 and 3.2.6 to nonconstant geometric transformations. An alternative that differentiates the original mapped function without intermediate projection is derived in Section A.1. That isolated experiment does not replace the recursive method described here.

3.4.1 What is represented, and what is differentiated?

The input to the derivative calculation is the DG function on one element, not the unknown exact solution of the conservation law. Its coefficient vector $U = (U_1, \dots, U_{N_k})^\top$ represents

$$u_h(x) = \sum_{b=1}^{N_k} U_b \varphi_b(x), \quad \hat{u}_h(\xi) = u_h(T_K(\xi)) = \sum_{b=1}^{N_k} U_b \hat{\varphi}_b(\xi). \quad (3.34)$$

The coefficients are the same in these two expressions. The physical basis is obtained by composing the reference basis with T_K^{-1} , so geometry changes the function's dependence on physical position without changing the meaning of the coefficient expansion. In particular, a polynomial in ξ need not be a polynomial in x .

All differentiations below are taken inside one element. We do not take a distributional derivative across the discontinuities of the DG solution. The separate element derivatives are evaluated at faces and their jumps are formed afterwards, as in Section 3.2.6. This distinction allows each element to have a well-defined smooth interior derivative even when the global DG function is discontinuous.

Reference differentiation is already available through $D_{\xi_r} = V^{-1}G^{(r)}$ from Section 3.2.5. The next question is how to turn this reference operation into a physical derivative when the element map is non-affine. We first identify the representation problem, then construct one projected derivative, and finally explain repeated differentiation.

3.4.2 Why the affine construction is insufficient

Let $x = T_K(\xi)$ be a smooth, invertible, orientation-preserving element map, and write

$$J_K(\xi) = \frac{\partial T_K}{\partial \xi}, \quad w_K(\xi) = \det J_K(\xi) > 0. \quad (3.35)$$

For $v = \hat{v} \circ T_K^{-1}$, change of variables and the chain rule give

$$\int_K v dx = \int_{\hat{K}} \hat{v} w_K d\xi, \quad (3.36)$$

$$(\nabla_x v) \circ T_K = J_K^{-T} \nabla_{\xi} \hat{v}. \quad (3.37)$$

The physical inner product therefore has a nonconstant weight. Even if $\hat{v}$ and T_K are polynomial, the inverse Jacobian generally makes $\partial_{x_j} v$ fall outside the mapped finite element space. Evaluating a single inverse Jacobian at the element centre cannot represent this variation. Componentwise, the chain rule reads

$$(\partial_{x_j} u_h)(T_K(\xi)) = \sum_{r=1}^d (J_K^{-1}(\xi))_{rj} \sum_{b=1}^{N_k} U_b \partial_{\xi_r} \hat{\phi}_b(\xi). \quad (3.38)$$

Thus the exact first physical derivative of the represented u_h can still be evaluated at any quadrature point using the available basis derivatives and Jacobian. The difficulty is storing that entire derivative with only N_k coefficients in the original space. On an affine map the geometric factors are constant and the representation is exact; on a curved map they are position-dependent and generally take the derivative outside the space.

3. METHODS

A one-dimensional example makes the obstruction explicit. On $[0, 1]$, take

$$x = T(\xi) = \xi + \varepsilon \xi(1 - \xi), \quad 0 < |\varepsilon| < 1, \quad \widehat{v}(\xi) = \xi. \quad (3.39)$$

Although the reference state is linear,

$$(\partial_x v) \circ T = \frac{1}{1 + \varepsilon - 2\varepsilon\xi}, \quad (\partial_x^2 v) \circ T = \frac{2\varepsilon}{(1 + \varepsilon - 2\varepsilon\xi)^3}. \quad (3.40)$$

Neither expression is generally a finite-degree reference polynomial. The issue is already present for an ordinary polynomial geometry map; it does not require NURBS.

Figure 3.4 shows the corresponding geometric issue in two dimensions: reference coordinate lines are mapped to a curved physical element, and the Jacobian weight and face normals vary within that element.

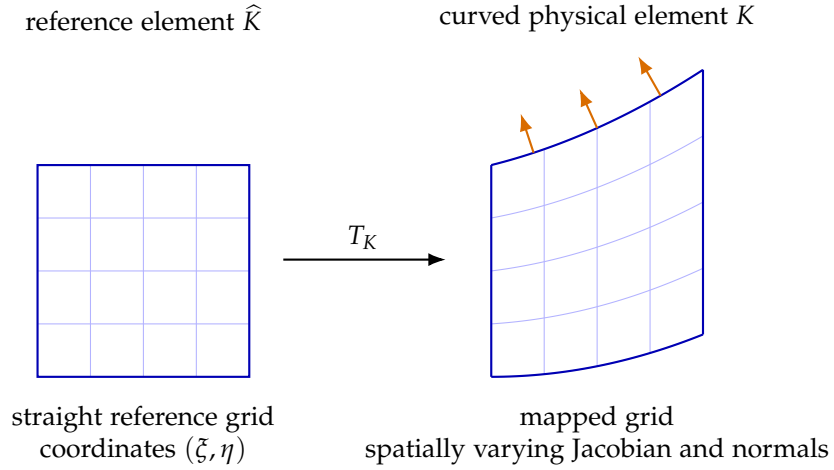


Figure 3.4: An illustrative polynomial map $T_K(\xi, \eta) = (\xi, (1 + \xi/4)\eta + \xi^2/5)$ from the unit square. Its determinant is $1 + \xi/4 > 0$: equal reference areas need not have equal physical areas. Orange arrows show outward normal directions along the curved top face (not their magnitudes). The interior lines are coordinate lines within one element, not separate DG cells. This is a geometric schematic, not a mesh from a numerical experiment.

3.4.3 Physical mass matrices and nested projections

Use the mapped spaces $V^r(K)$ from Eq. (2.2), all with the same map T_K . Let $\widehat{\varphi}_i$ span the order- k reference space and $\widehat{\psi}_a^{(r)}$ its order- r subspace. Their physical mass and coupling matrices are defined below. Let N_k and N_r be the numbers of basis functions at orders k and r . Then M_k^K is $N_k \times N_k$, M_r^K is $N_r \times N_r$, and the coupling matrix B_r^K is $N_r \times N_k$. Indices i, j run over the

order- k basis and a, b over the order- r basis:

$$(M_k^K)_{ij} = \int_{\hat{K}} \hat{\varphi}_i \hat{\varphi}_j w_K d\hat{\xi}, \quad (3.41)$$

$$(M_r^K)_{ab} = \int_{\hat{K}} \hat{\psi}_a^{(r)} \hat{\psi}_b^{(r)} w_K d\hat{\xi}, \quad (3.42)$$

$$(B_r^K)_{ai} = \int_{\hat{K}} \hat{\psi}_a^{(r)} \hat{\varphi}_i w_K d\hat{\xi}. \quad (3.43)$$

If U represents u_h , its physical L^2 projection into $V^r(K)$ has lower-order coefficients C_r determined by

$$M_r^K C_r = B_r^K U. \quad (3.44)$$

This is the usual finite element projection equation, now with the physical Jacobian weight retained. To express the projected function in the original order- k basis, solve $M_k^K U_r = (B_r^K)^\top C_r$. Thus the matrix representing Π_K^r in that basis is

$$P_r^K = (M_k^K)^{-1} (B_r^K)^\top (M_r^K)^{-1} B_r^K. \quad (3.45)$$

The inverse notation denotes the associated linear solves. Nesting makes this lifting exact as a function-space operation. With exact integrals,

$$P_r^K P_s^K = P_{\min(r,s)}^K, \quad (P_r^K)^\top M_k^K = M_k^K P_r^K. \quad (3.46)$$

Consequently the physical orthogonal shells and the frozen exponential attenuation in Eq. (3.27) remain available. No orthogonal storage basis is required.

Mean preservation follows directly from testing the projection equation with one: $\int_K (\Pi_K^r u_h - u_h) dx = 0$. In coefficient form, let $b_i = \int_K \varphi_i dx$; then $b^\top P_r^K = b^\top$. Every nonconstant shell has zero b -weighted integral, so damping it leaves $b^\top U$ unchanged component by component. On affine elements, w_K is constant and cancels from Eq. (3.45), recovering the reusable reference projector. On curved elements it cannot be cancelled.

3.4.4 Projecting a physical derivative

Let $f = \partial_{x_j} u_h$ denote the exact physical derivative of the represented input. Since f need not belong to $V^k(K)$, we select the representable function with the smallest physical squared-integral error:

$$z_h = \operatorname{argmin}_{z \in V^k(K)} \int_K |z - f|^2 dx = \Pi_K^k f = \mathcal{D}_{K,j} u_h. \quad (3.47)$$

The word physical refers to the measure dx . A region stretched by the map occupies more physical volume and must have correspondingly more

3. METHODS

weight in this error. Minimizing an unweighted reference integral would generally choose a different approximation.

To obtain the minimizer, require the first variation in every available test direction v_h to vanish. This gives the orthogonality condition

$$(z_h, v_h)_K = (\partial_{x_j} u_h, v_h)_K \quad \text{for every } v_h \in V^k(K). \quad (3.48)$$

The residual $z_h - f$ is therefore orthogonal to the available approximation space. This does not require it to vanish at nodes or face points: physical L^2 projection is an integral best approximation, not interpolation. Write the unknown projected derivative as $z_h = \sum_b Z_b \varphi_b$ and test with each φ_a . The result is $\sum_b (\varphi_a, \varphi_b)_K Z_b = \sum_b (\varphi_a, \partial_{x_j} \varphi_b)_K U_b$. The left-hand side is the physical mass matrix acting on Z ; the right-hand side supplies the inner products with the known derivative. Using Eq. (3.37) gives

$$M_k^K Z = G_{K,j} U, \quad D_{K,j} = (M_k^K)^{-1} G_{K,j}, \quad (3.49)$$

$$(G_{K,j})_{ab} = \int_{\hat{K}} \hat{\varphi}_a \left[\sum_{s=1}^d (J_K^{-1})_{sj} \partial_{\xi_s} \hat{\varphi}_b \right] w_K d\xi. \quad (3.50)$$

Here $U, Z \in \mathbb{R}^{N_k}$ and M_k^K , $G_{K,j}$, and $D_{K,j}$ are all $N_k \times N_k$. The subscript j selects one physical coordinate direction. The integral matrix $G_{K,j}$ must not be confused with the sampled reference-derivative matrix $G^{(r)}$ used to construct D_{ξ_r} . The matrix $D_{K,j}$ maps input coefficients directly to projected derivative coefficients: $Z = D_{K,j} U$.

At each volume quadrature point, assembly evaluates the reference basis, its first derivatives, the inverse Jacobian, and the physical weight. It accumulates the mass and derivative inner products, then solves the mass system for each column of $G_{K,j}$. The static geometry allows the resulting $D_{K,j}$ to be stored and reused. During filtering, applying a derivative is a matrix multiplication; no new global PDE problem is solved.

The geometry is evaluated inside the integral. Only first reference basis derivatives and first derivatives of the map are needed. A constant has zero right-hand side, hence zero projected derivative. For affine maps, physical differentiation stays in the supported space and this projection recovers the existing exact polynomial derivative matrix.

In the one-dimensional example Eq. (3.40), the projected first derivative z_h is characterized by

$$\int_0^1 \hat{z}_h \hat{\varphi}_a (1 + \varepsilon - 2\varepsilon\xi) d\xi = \int_0^1 \hat{\varphi}_a d\xi. \quad (3.51)$$

The Jacobian cancels on the right, but remains in the mass matrix on the left. This illustrates why a physical projection is different from an unweighted

reference projection. To make the result explicit, set $\varepsilon = 1/2$ and use the reference linear basis $(1, \xi)$. Then

$$\begin{pmatrix} 1 & 5/12 \\ 5/12 & 1/4 \end{pmatrix} \begin{pmatrix} Z_0 \\ Z_1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1/2 \end{pmatrix}, \quad \hat{z}_h(\xi) = \frac{6}{11} + \frac{12}{11}\xi. \quad (3.52)$$

This line approximates $1/(3/2 - \xi)$ in the physical integral norm. At $\xi = 1$ it has value $18/11$, whereas the exact derivative is 2. The mismatch is compatible with the best-approximation property: endpoint agreement was never imposed. This intentionally coarse example illustrates the definition, rather than predicting the error of a refined simulation.

In actual assembly the integrals are replaced by quadrature, giving the projection associated with that discrete inner product. Increasing quadrature accuracy reduces integration error, but cannot make an unrepresentable derivative belong to the fixed solution space.

3.4.5 Recursive higher derivatives: the precise approximation

After the first operation, only the coefficients of $z_x = \Pi_K^k \partial_x u_h$ are retained as the derivative state. A second x -operation differentiates this represented approximation and projects again:

$$z_x = \Pi_K^k \partial_x u_h, \quad z_{xx} = \Pi_K^k \partial_x z_x = \Pi_K^k \partial_x (\Pi_K^k \partial_x u_h). \quad (3.53)$$

It does not return to u_h and evaluate its exact second physical derivative. The distinction persists even if both projections are assembled with exact integration. The derivative graph implements this sequence by reusing the same first-derivative matrices:

$$U^{(0)} = U, \quad U^{(\alpha)} = D_{K,j} U^{(\alpha - e_j)}, \quad (3.54)$$

For $\alpha \neq 0$, the code chooses $j = \max\{s : \alpha_s > 0\}$. The coordinate unit multi-index e_j has a one in position j and zeros elsewhere, so $\alpha - e_j$ denotes the already computed parent state with one fewer derivative. $U^{(\alpha)}$ is the coefficient vector of $\mathcal{D}_K^\alpha u_h$, not a power of U . For $(1,1)$, for example, the parent is $(1,0)$ and the last operation is $D_{K,y}$; for $(2,1)$, the chain is $(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (2,1)$. This gives exactly the ordering in Eq. (3.11). For instance, the second operation in directions i then j represents $\Pi_K^k \partial_{x_j} (\Pi_K^k \partial_{x_i} u_h)$. It generally differs from projecting the exact second derivative once. Writing $\Pi = \Pi_K^k$,

$$\Pi \partial_{x_j} (\Pi \partial_{x_i} u_h) - \Pi \partial_{x_j} \partial_{x_i} u_h = \Pi \partial_{x_j} (\Pi - I) \partial_{x_i} u_h. \quad (3.55)$$

To read this identity, set $e_i = (\Pi - I) \partial_{x_i} u_h$. The first stored derivative is $\partial_{x_i} u_h + e_i$, so the next operation is $\Pi \partial_{x_j} \partial_{x_i} u_h + \Pi \partial_{x_j} e_i$. The right-hand side

of Eq. (3.55) is precisely this last term. A small error in the first projection does not by itself guarantee a small higher-derivative error, because differentiating the error can amplify it. There are also two distinct accuracy questions: error relative to derivatives of the represented u_h , and error relative to derivatives of an underlying smooth exact solution u . The latter includes differentiation of the input approximation error $u_h - u$ as well as the intermediate projections. It explains the difference between successive projection and a single projection of the exact derivative. To isolate the ordering issue, subtract the same identity with i and j interchanged. Exact mixed partials cancel:

$$(\mathcal{D}_{K,j}\mathcal{D}_{K,i} - \mathcal{D}_{K,i}\mathcal{D}_{K,j})u_h = \Pi\partial_{x_j}(\Pi - I)\partial_{x_i}u_h - \Pi\partial_{x_i}(\Pi - I)\partial_{x_j}u_h. \quad (3.56)$$

Thus any failure to commute comes from the intermediate projection errors, not from a failure of the physical chain rule on curved geometry. The two terms vanish when the first derivatives are represented exactly in the space. On curved elements they need not vanish, although particular states or maps can still give commuting results.

The derivative graph stores one state per multi-index. On the affine path, any valid parent gives the same exact derivative; the curved extension retains one fixed parent as a definition of the approximation. It neither computes every ordering nor verifies that alternative parents agree. A multi-index remains a valid storage label with this convention, but it is not evidence of order independence. Averaging derivative permutations or projecting exact higher derivatives would define different sensor operators and would require separate implementation and validation. Second- and third-derivative convergence must therefore be assessed using manufactured solutions. Projection alone is not a proof of their convergence.

The two routes are contrasted in Fig. 3.5.

3.4.6 Curved faces, normal directions, and length scales

For a reference face with outward unit normal $\hat{n}$, the physical surface factor and outward normal are

$$s_F = \|w_K J_K^{-T} \hat{n}\|, \quad \mathbf{n}_K = \frac{w_K J_K^{-T} \hat{n}}{s_F}, \quad ds = s_F d\hat{s}. \quad (3.57)$$

These quantities vary along a curved face. For face quadrature weights ω_q , use

$$|F|_Q = \sum_q \omega_q s_{F,q}, \quad \tilde{\omega}_q = \frac{\omega_q s_{F,q}}{|F|_Q}, \quad \sum_q \tilde{\omega}_q = 1. \quad (3.58)$$

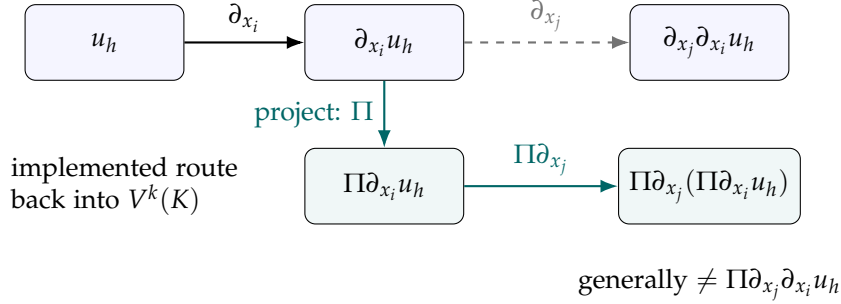


Figure 3.5: Exact differentiation and the implemented recursive construction on a curved element, with $\Pi = \Pi_K^k$. The first physical derivative generally leaves $V^k(K)$. Projecting it back changes the input to the next derivative. The lower route therefore differs from differentiating twice and projecting only at the end. Each lower state is represented in the same finite element space; no second derivative of the geometry map is needed by this route.

Hence the implemented derivative-jump average is

$$J_{F,c}^{(l)} \approx \sum_{|\alpha|=l} \sum_q \tilde{\omega}_q \left| (\mathcal{D}_{K^-}^\alpha u_{h,c}^-)(x_q) - (\mathcal{D}_{K^+}^\alpha u_{h,c}^+)(x_q) \right|. \quad (3.59)$$

Both traces are evaluated at the same physical point, using their respective element maps. A constant jump has the same normalized magnitude on a straight or curved face. The one-sided wave radii use $\mathbf{n}_K(x_q)$ at these points, rather than a single centre normal.

Volume quadrature gives $|K|_Q = \sum_q \omega_q w_K(\xi_q)$ and the OFDG length $h_K = (|K|_Q / |\hat{K}|)^{1/d}$. The global mean and amplitude calculation uses the same physical volume interpretation. KXRCF uses its separately defined sampled radius and restricts its signed face integral to quadrature points with incoming transport. Its denominator uses the corresponding physical inflow measure. Thus sensing, normalization, and propagation directions all refer to the curved physical geometry.

The algebra above uses exact integrals. The implementation assembles them with positive, overintegrated quadrature. Consistent quadrature on the nested spaces gives the analogous discrete projection identities, up to linear-solve roundoff; it preserves the quadrature-defined physical means. Agreement with exact integrals, derivative accuracy, and geometric sampling invariance still require quadrature and refinement checks. This solution-space projection does not convert rational NURBS geometry to polynomial geometry; that separate, explicit geometry approximation retains the support boundary stated in Section 3.8.

3.5 The 2024 OEDG comparator

OEDG is a related projection-based damping method, published online by Peng, Sun, and Wu in 2024 [4]. This section describes the affine implementation available for comparison with adapted OFDG. Its sensor, component pooling, and time integration define the differences between the methods. Whereas earlier multidimensional OFDG sensors evaluate vertex jumps [3, 2], OEDG uses an interface integral of absolute jumps. A paper-faithful curvilinear OEDG extension is not established here.

3.5.1 Scale-invariant interface sensor

Let K be an element, $e \subset \partial K$ a face, k the DG degree, and $m \in \{0, \dots, k\}$. For component i , define the global amplitude

$$A_i = \left\| u_h^{(i)} - \text{avg}_\Omega(u_h^{(i)}) \right\|_{L^\infty(\Omega)}. \quad (3.60)$$

The maximum is evaluated at the same volume quadrature nodes used by the filter. When A_i is below a relative machine-scale tolerance, all associated sensors are zero. The remaining geometric ingredient is the face height $h_{e,K} = \sup_{x \in K} \text{dist}(x, e)$, the greatest distance from the element to face e . Here $|e|$ is the physical face measure, ∂^α is the exact derivative from Eq. (3.9), and m denotes total derivative order in this section, rather than the number of system components. With these ingredients, the face contribution corresponding to equation (3.10) of [4] is

$$\sigma_{e,K}^{m,(i)} = \frac{(2m+1)h_{e,K}^m}{2(2k-1)m! A_i} \sum_{|\alpha|=m} \frac{1}{|e|} \int_e \left| \llbracket \partial^\alpha u_h^{(i)} \rrbracket \right| dS, \quad (3.61)$$

The implementation computes this face height geometrically and caches it; on a simplex it is the altitude associated with e , not a generic element diameter. In 2D the face average uses the endpoint trapezoidal rule employed in the paper. Point faces in 1D require no quadrature choice. The present 3D path retains a proper high-order face rule because the paper specifies the trapezoidal treatment only for 2D computations.

For a system, the paper avoids characteristic decomposition and pools physical components,

$$\sigma_{e,K}^m = \max_i \sigma_{e,K}^{m,(i)}. \quad (3.62)$$

The damping rate combines all adjacent faces,

$$\delta_K^m = \sum_{\substack{e \subset \partial K \\ e \in \mathcal{F}_h^1}} \frac{\beta_{K,e}}{h_{e,K}} \sigma_{e,K}^m, \quad (3.63)$$

where $\beta_{K,e}$ is the one-sided local normal spectral radius. Physical boundary faces are excluded in this implementation; boundary conditions enter the DG residual. Thus advection uses $|a \cdot n|$, Burgers uses the corresponding normal flux derivative, and Euler uses $|v \cdot n| + c$. Following the paper, state-dependent speeds are evaluated from the adjacent element means rather than from face traces; spatially varying prescribed velocities and normals are still sampled along the face.

The normalized sensor is unchanged under a nonzero scaling and constant shift when the amplitude cutoff is inactive: numerator and denominator scale together. This sensor statement must be separated from the complete filter, whose state-dependent wave speeds can change. With fixed unit speed, the attenuation inherits the same scale/shift equivariance away from the cutoff.

Evolution invariance concerns rescaling the flux speed and physical time, not adding a constant state. Multiplying the flux and its speeds by $a > 0$ while replacing the damping duration by τ/a preserves the product of rate and duration. The paper develops these properties under its stated assumptions [4]. No general invariance under arbitrary nonlinear state transformations is asserted here.

3.5.2 Exact attenuation without a prescribed coefficient basis

Let P^m denote the element-local L^2 projection into the nested order- m reference family mapped to the element, with $P^{-1} = P^0$. During the OE pseudo-time problem, the coefficients Eq. (3.63) are frozen at the candidate RK stage. The published derivation may be written using coefficients of a local orthogonal basis, with scaled Legendre polynomials as one convenient one-dimensional choice. Such a basis is not mathematically required. In basis-independent operator form, the exact attenuation is

$$\mathcal{F}_{\tau,K} u_h = P^0 u_h + \sum_{j=1}^k \exp\left(-\tau \sum_{m=0}^j \delta_K^m\right) (P^j - P^{j-1}) u_h, \quad (3.64)$$

so the constant subspace is unchanged and every order- j shell is scaled by its corresponding exponential. The implementation assembles the projectors in MFEM's supplied coefficient basis and applies Eq. (3.64) without ever forming Legendre coefficients. No pseudo-time discretization error is introduced, and every quadrature-weighted physical element mean is preserved to roundoff.

The public OEDG2024 type fixes the face quadrature, face-height, per-face component pooling, relative constant-state test, mean-state wave speed, normalization, and exact-decay choices. It reuses the already tested projection, derivative, immutable face cache, and MPI halo machinery of the adapted

OFDG kernel. This sharing removes duplicate geometry code without allowing the comparator to inherit KXRCF or an experiment-only option accidentally. It also gives OFDG and OEDG the same basis-agnostic projection realization, preventing a change of coefficient basis from becoming an uncontrolled difference in the numerical comparison.

3.5.3 Runge–Kutta time integration

The OEDG algorithm applies its damping map after each Runge–Kutta stage. The default adapted OFDG configuration applies one damping map after a full step. Both cadences are exposed by the implementation for controlled numerical comparisons. Consequently, a difference between the default methods cannot be attributed solely to their sensors. A final experimental comparison must state the cadence and distinguish it from the projection and normalization choices. The present account restricts the OEDG comparator to affine elements.

3.6 Numerical controls

3.6.1 Euler admissibility

Oscillation damping does not guarantee positive density and pressure. The implementation therefore includes a separate control for Euler states. For an element mean $\bar{u}_K$, it replaces a candidate polynomial by

$$u_h^{\text{lim}} = \bar{u}_K + \text{heta}_K(u_h - \bar{u}_K), \quad 0 \leq \theta_K \leq 1. \quad (3.65)$$

A single factor is used for all conservative components. Integration shows that the component means are unchanged. The factor is restricted first for density, then for pressure by bisection along the segment toward the mean. Admissibility is checked at the volume and face points needed by the evolution and filtering operators; it is not a statement about every point of the continuous polynomial.

A nonphysical mean cannot be repaired by scaling toward itself. In that case the candidate step is rejected and retried with a smaller timestep, subject to a retry limit. Comparisons between Euler methods must apply the same admissibility control and record its interventions so that its influence is not attributed to the oscillation filter.

[PLACEHOLDER POSITIVITY-SOURCE — Still needs:]

Review the proposed positivity-preserving DG reference before adding its citation and relating this implementation to the published assumptions. The scaling formula above describes the implemented control and does not claim a positivity theorem for the complete scheme.

3.6.2 Experimental choices still to be fixed

The formulation distinguishes approximation order, spatial resolution, CFL number, filtering cadence, detector threshold, and positivity treatment. These must be specified when comparing methods. In particular, comparing post-step OFDG with stage-wise OEDG changes both the sensor and the placement of filtering. Such a comparison cannot isolate the sensor's effect without an additional controlled comparison of cadences.

The final selection of problems, reference solutions, error measures, and resolution studies is deferred to the Results chapter. Existing exploratory calculations and their reference-solver workflow remain separate from this methodological account.

3.7 Putting the OFDG building blocks together

The following sequence specifies one application of the default post-step OFDG filter. Its inputs are the candidate DG coefficients U^* , the fixed mesh and solution space, a damping duration Δt , and the chosen wave-speed policy. Physical projection and derivative matrices and geometric quadrature data have already been assembled as in Sections 3.4 and 3.8.

1. If KXRCE is enabled, evaluate the component indicators from U^* and form the common active set $\mathcal{A}$ using Eq. (3.31). Otherwise every element is active. An inactive element will retain its candidate coefficients exactly.
2. Evaluate the physical global mean and sampled amplitude A_c for each component, using Eq. (3.22). These statistics include all elements, even when filtering is restricted to an active set. Initialize every element rate to zero; components with $A_c \leq 10^{-14}$ receive no damping.
3. Starting from U^* , compute the derivative states for all $0 \leq |\alpha| \leq k$ with the parent rule in Eq. (3.54). Include neighbour states needed at shared interfaces. The state for $\alpha = 0$ is the solution itself.
4. On each interior face needed by an active element, evaluate both traces at matching physical quadrature points. Form the absolute-jump averages Eq. (3.19) with normalized physical weights, and evaluate each side's normal wave speed. Add its contribution from Eq. (3.23) to the corresponding element rate. A face adjacent to an inactive neighbour still contributes to an active element.
5. Hold these rates fixed. For each active element and component, retain $\Pi_K^0 u_{h,c}^*$ and multiply each shell $(\Pi_K^j - \Pi_K^{j-1}) u_{h,c}^*$ by $\exp(-\Delta t \sum_{l=0}^j \delta_{K,c}^{(l)})$. Sum the retained mean and attenuated shells to obtain the filtered coefficients.

All indicators, amplitudes, derivatives, and rates in this application refer to the same candidate state. Rates are not recomputed between shells. The result is the map $\mathcal{F}_{\Delta t}(U^*)$ introduced at the start of this chapter. Euler admissibility remains the separate control described in Section 3.6; OEDG instead uses its own sensor and the stage-wise placement described in Section 3.5. The selected mesh, initial and boundary data, CFL number, and final time must additionally be specified for a reproducible experiment; their final selection remains pending, rather than being implicit in this filter definition.

3.8 Implementation assumptions and geometric scope

The implementation separates equation-dependent face physics, geometry-dependent operators, and damping. MFEM supplies meshes, finite element bases, geometric transformations, and distributed ownership. This organisation allows the same projection algebra to act on scalar fields and conservative systems.

The finite element space must be nonempty and discontinuous; the caller is responsible for its validity and for rebuilding the filter after mesh changes. The public OFDG interface provides construction, stabilization, and decay; KXRFC supplies an independent optional activation mask. The supported construction assumes full-dimensional elements, a fixed mesh, and uniform solution order $k \geq 1$. It covers affine intervals and polynomial curved triangles, quadrilaterals, tetrahedra, hexahedra, and prisms. Different supported element families may coexist at the same order. Moving meshes, variable solution order, pyramids, and embedded surfaces are excluded.

Native NURBS geometry is not supported by the two-sided face path used with the project's pinned MFEM version. A NURBS mesh may instead be explicitly approximated by polynomial geometry before the filter is constructed. This changes the geometric representation and is separate from the solution-space L^2 projection derived above. It does not imply that MFEM lacks NURBS support in general. The OEDG implementation is treated here as an affine comparator; the curvilinear extension described in this chapter concerns adapted OFDG and KXRFC.

3.8.1 Reference data and physical operators

Reference basis evaluations and the derivative graph are reusable by finite element signature. Physical mass, projection, and derivative matrices belong to each curved element. The reference projector is sufficient only when the constant affine Jacobian weight cancels. Element and face geometry data remain fixed during the lifetime of a filter; a change in mesh geometry, topology, or solution space requires reconstruction.

Curved integration uses quadrature order $2k + 2 \max(0, \text{OrderW}) + 8$, where MFEM's `OrderW` estimates the transformation-weight order. Positive tensor or Duffy quadrature and symmetric triangular sampling are used to reduce orientation-dependent sampling effects. This is an integration policy, not a guarantee of exact integration for every mapped function. The physical projection identities refer to the resulting quadrature inner product unless exact integration is stated explicitly.

The implementation retains derivative coefficient states for every local cell and rebuilds them for each new solution. For gated filtering, local faces whose two neighbours are inactive are skipped before their geometry is constructed. Adjacent damping shells reuse their shared projection. These choices eliminate redundant work without changing the operator. The matrix construction is given in Sections 3.2.3, 3.2.5 and 3.2.6; the storage costs are specified in Section 3.8.3.

3.8.2 Distributed evaluation

MPI distributes elements across processes, or ranks. An ordinary interior face contributes to both adjacent elements. At a shared face, a rank imports neighbour coefficients and accumulates only its own element's contribution; the adjacent rank performs the complementary operation. Global solution amplitudes require collective reductions. Local and imported elements use the same physical operator construction, so an MPI interface has no additional physical boundary condition.

Geometry caches are fixed, but their integration-point state and the working arrays are mutable. Concurrent applications of the same filter object are therefore unsupported. This ownership convention keeps numerical state separate from reusable geometric data.

3.8.3 Runtime data and storage costs

Affine volume means and sampled extrema use quadrature of order $2k + 1$; curved elements use the geometry-dependent rule in Section 3.8.1. These are the samples used for A_c in Eq. (3.22), including its stated small-amplitude cutoff. The public semidiscrete interface uses the same rates as exponential decay, but returns the coefficient vector of Eq. (3.24).

Derivative coefficient storage is allocated persistently for every local element and rebuilt for each candidate state. Each interior face stores its two evaluation matrices and normalized quadrature weights. There is no cache eviction or replacement policy in the derivative or face loop. To make the memory implications explicit, let $N_k(K)$ be the number of coefficients on element K and $N_{q,F}$ the number of face quadrature points. The derivative graph contains $\binom{k+d}{d}$ states per component and element. The corresponding storage

scales as

$$\mathcal{O}\left(\sum_{K \in \mathcal{T}_h} dN_k(K)^2\right) \quad \text{for physical derivative matrices,} \quad (3.66)$$

$$\mathcal{O}\left(\sum_{K \in \mathcal{T}_h} mN_k(K) \binom{k+d}{d}\right) \quad \text{for derivative coefficient states,} \quad (3.67)$$

$$\mathcal{O}\left(\sum_{F \in \mathcal{F}_h^i} N_{q,F} [N_k(K^-) + N_k(K^+) + 1]\right) \quad \text{for face evaluation data.} \quad (3.68)$$

The sums allow mixed element families with different coefficient counts. On one MPI rank they describe its local elements and ordinary interior faces; shared-face neighbour buffers require additional storage. These estimates cover the listed arrays, not the entire solver: projection matrices, geometric data, and runtime scratch arrays also occupy memory. Reusable affine operators avoid duplicating reference data, whereas curved physical projection matrices must be assembled per element. This storage policy trades memory for reuse of fixed operators and avoids rebuilding them during every timestep.

3.8.4 Verification criteria

The algebra suggests independent checks: constants and component-wise cell means should be preserved; inactive cells should remain unchanged; and a change of coefficient basis should represent the same filtered function. Curved first derivatives can be compared with an independently assembled physical projection. Recursive higher derivatives require manufactured-solution refinement studies, since their definition alone supplies no convergence rate. Quadrature sensitivity, geometric parameterization, and serial/MPI agreement are separate checks of the geometric realization.

These checks distinguish consistency of an implementation from the accuracy and robustness of a complete simulation. Their selected numerical evidence, along with comparisons between methods, belongs in the Results chapter once the experimental programme has been agreed.

Chapter 4

Results

[PLACEHOLDER RESULTS — Still needs:]

Supervisor-approved experiment selection; final configurations and reference solutions; validated figures, tables, and uncertainty or sensitivity checks.

Chapter 5

Discussion

[PLACEHOLDER DISCUSSION — Still needs:]

Interpretation of the agreed results against the research questions; limitations and alternative explanations; comparison with the cited literature.

Chapter 6

Conclusion

[PLACEHOLDER CONCLUSION — Still needs:]

Evidence-supported answers to the research questions; final statement of contributions and scope; justified directions for future work.

Appendix A

Direct curved differentiation and surface geometry

This appendix records the mathematics behind two isolated investigations: direct physical differentiation of a mapped DG function, and first-order OFDG on a spherical surface. They answer different questions. The first changes how derivatives of a function on a volume element are evaluated. The second changes the domain from a volume to a two-dimensional surface embedded in three-dimensional space. A curved volume element remains a full-dimensional region; a surface element does not.

SCOPE LIMITATION

The production method remains the recursively projected volume construction in Section 3.4. Direct differentiation was investigated in a separate operator experiment. The surface implementation is an isolated P1/Q1 rotational-transport example, using only first surface derivatives. Higher-order surface OFDG and general production surface support are outside the scope of this thesis. The final section explains that boundary; it does not specify an implemented extension.

The derivations below introduce their building blocks before combining them. In particular, a coordinate transformation, a physical L^2 projection, and a geometric projection onto a tangent plane are three distinct operations. Numerical comparisons remain in the separate supervisor brief; this appendix does not turn preliminary operator measurements into a PDE accuracy claim.

A.1 Direct physical differentiation of the original DG function

A.1.1 The function is already defined in physical space

Let $K \subset \mathbb{R}^d$ be a volume element and let $T : \hat{K} \rightarrow K$ be a smooth invertible map. The reference coordinate is $\xi = (\xi_1, \dots, \xi_d)$ and the physical coordinate is $x = (x_1, \dots, x_d) = T(\xi)$. Write $S = T^{-1}$ for the inverse map. For coefficients U_b and reference basis functions $\hat{\varphi}_b$,

$$\hat{u}_h(\xi) = \sum_{b=1}^N U_b \hat{\varphi}_b(\xi), \quad u_h(x) = \hat{u}_h(S(x)). \quad (\text{A.1})$$

The second expression is the physical function, not an approximation to a second representation of it. Its physical basis is simply $\varphi_b(x) = \hat{\varphi}_b(S(x))$. Thus there is no need to project the input into a new space merely to regard it as a function of physical position. The original coefficients already define that function exactly, subject to floating-point evaluation and the chosen geometry representation.

On a curved element the φ_b generally are not physical polynomials. They remain legitimate basis functions. We can evaluate and differentiate such compositions without requiring their derivatives to lie in the original finite-dimensional space. What we lose is the ability to store an arbitrary exact derivative using just another coefficient vector in that same space. Pointwise evaluation and finite-element representation are therefore different requirements.

This distinction also separates the present experiment from reconstructing u_h as a polynomial in x . A physical-polynomial L^2 reconstruction would usually change the input function before differentiation. Direct composition with S does not. All derivatives in this section are element-interior classical derivatives. We do not differentiate the discontinuous global DG function in the distributional sense.

A.1.2 First derivatives: why the Jacobian is still needed

Define $J_{ar} = \partial T_a / \partial \xi_r$ and $A_{ri} = (J^{-1})_{ri}$. Indices a, i, j refer to physical coordinates; r, s refer to reference coordinates. The inverse function identity $T(S(x)) = x$ gives $DS(x) = J(S(x))^{-1}$. The chain rule then yields

$$\partial_{x_i} u_h(x) = \sum_{r=1}^d \partial_{\xi_r} \hat{u}_h(S(x)) A_{ri}(S(x)). \quad (\text{A.2})$$

The entries of A describe how the reference position changes when one physical coordinate changes. Mapping the function to physical space has

not removed this dependence: it has placed it inside the composition in Eq. (A.1). If we differentiate the whole composition symbolically or by automatic differentiation, the chain rule supplies these factors automatically. If we differentiate the reference polynomial by hand, we must supply them explicitly. Omitting them would differentiate a different function with respect to a different coordinate.

A simple scaling illustrates this. If $x = 2\xi$ and $\hat{u}_h(\xi) = \xi$, then $u_h(x) = x/2$. Its physical derivative is $1/2$, whereas the reference derivative is 1 . The inverse Jacobian is precisely the missing factor. Curvature adds variation of that factor with position.

A.1.3 Second derivatives: differentiate the geometric factor too

The Hessian of a scalar function is the matrix of all its second partial derivatives. In two dimensions it is

$$H_x u_h = \begin{pmatrix} \partial_{xx} u_h & \partial_{xy} u_h \\ \partial_{yx} u_h & \partial_{yy} u_h \end{pmatrix}. \quad (\text{A.3})$$

This is notation for familiar derivatives, rather than a new differentiation operation. Applying the product and chain rules to Eq. (A.2) gives, with all reference quantities evaluated at $\xi = S(x)$,

$$\partial_{x_j} \partial_{x_i} u_h = \sum_{r,s=1}^d A_{sj} A_{ri} \partial_{\xi_s} \partial_{\xi_r} \hat{u}_h + \sum_{r,s=1}^d A_{sj} (\partial_{\xi_s} A_{ri}) \partial_{\xi_r} \hat{u}_h. \quad (\text{A.4})$$

The first sum differentiates the reference function. The second differentiates the geometric factor in its first physical derivative. On an affine element A is constant, so the second sum vanishes. On a curved element it generally does not. Applying the first-derivative transformation twice while treating A as constant misses exactly this contribution.

The required inverse-Jacobian derivative follows by differentiating $JA = I$:

$$\partial_{\xi_s} A = -A(\partial_{\xi_s} J)A. \quad (\text{A.5})$$

Thus a direct second derivative needs second derivatives of the geometry, in addition to first and second derivatives of the reference solution. This requirement is different from the production recursion, which uses first geometry derivatives to assemble each projected first-derivative matrix and then differentiates the represented projection again.

A.1.4 A one-dimensional example through the third derivative

Consider

$$x = T(\xi) = \xi + \varepsilon \xi(1 - \xi), \quad 0 < |\varepsilon| < 1, \quad j(\xi) = T'(\xi) > 0. \quad (\text{A.6})$$

Let $f(\xi) = \hat{u}_h(\xi)$. Since $d/dx = j^{-1}d/d\xi$, repeated application of the product rule gives

$$u'_h = \frac{f'}{j}, \quad (\text{A.7})$$

$$u''_h = \frac{f''}{j^2} - \frac{f'j'}{j^3}, \quad (\text{A.8})$$

$$u'''_h = \frac{f'''}{j^3} - \frac{3f''j'}{j^4} - \frac{f'j''}{j^4} + \frac{3f'(j')^2}{j^5}. \quad (\text{A.9})$$

Primes on f and j mean reference derivatives; primes on u_h mean physical derivatives. These equations are evaluated at $x = T(\xi)$. For $f(\xi) = \xi$, the reference second and third derivatives vanish, but $j = 1 + \varepsilon - 2\varepsilon\xi$, $j' = -2\varepsilon$, and $j'' = 0$ give

$$u'_h = j^{-1}, \quad u''_h = 2\varepsilon j^{-3}, \quad u'''_h = 12\varepsilon^2 j^{-5}. \quad (\text{A.10})$$

A reference-linear function can therefore have nonzero physical derivatives of every order. This is not numerical error: it is a property of the represented function on the curved element. The powers of j^{-1} also explain why nearly singular maps can amplify derivatives and numerical error.

A.1.5 Higher derivatives from a local inverse Taylor series

Writing every product-rule term separately becomes cumbersome in several dimensions. The isolated experiment instead stores a short multivariate Taylor polynomial, often called a *Taylor jet*. This polynomial is local to one evaluation point; it is not a new DG approximation or an L^2 projection of the solution.

Choose a reference point ξ_0 and its physical image $x_0 = T(\xi_0)$. Let $h > 0$ be a length scale and introduce a dimensionless physical increment $\eta = (\eta_1, \dots, \eta_d)$ by $x = x_0 + h\eta$. We seek the Taylor polynomial $q(\eta)$ of the inverse map about this point, through total degree m , satisfying

$$q(0) = \xi_0, \quad T(q(\eta)) = x_0 + h\eta + \mathcal{O}(|\eta|^{m+1}). \quad (\text{A.11})$$

Substitute it into the original reference function and retain terms through that degree:

$$\hat{u}_h(q(\eta)) = \sum_{|\alpha| \leq m} c_\alpha \eta^\alpha + \mathcal{O}(|\eta|^{m+1}). \quad (\text{A.12})$$

Here a multi-index is a tuple of nonnegative integers $\alpha = (\alpha_1, \dots, \alpha_d)$. Its total order, factorial and monomial are

$$|\alpha| = \sum_{i=1}^d \alpha_i, \quad \alpha! = \prod_{i=1}^d \alpha_i!, \quad \eta^\alpha = \prod_{i=1}^d \eta_i^{\alpha_i}. \quad (\text{A.13})$$

For example, $\alpha = (2, 1)$ means two x derivatives and one y derivative; $|\alpha| = 3$ and $\alpha! = 2$. Comparing with the Taylor expansion in physical coordinates shows that

$$\partial_x^\alpha u_h(x_0) = \frac{\alpha!}{h^{|\alpha|}} c_\alpha. \quad (\text{A.14})$$

The factorial and length-scale factors are essential. A Taylor coefficient is not in general equal to the derivative it encodes.

Polynomial addition is coefficientwise. Multiplication multiplies all pairs of monomials and discards products whose total degree exceeds m . Terms of higher degree cannot contribute to lower-degree coefficients under these operations. Consequently this truncation preserves the derivatives through order m at the base point in exact arithmetic. It does not promise an accurate approximation over a finite neighbourhood of arbitrary size.

For two dimensions and $m = 3$, ten coefficients suffice: one constant, two linear, three quadratic and four cubic coefficients. The experiment computes the inverse series using the fixed base-point Jacobian $J_0 = J(\xi_0)$:

$$q^{(0)}(\eta) = \xi_0, \quad (\text{A.15})$$

$$q^{(\ell+1)}(\eta) = \left[q^{(\ell)}(\eta) - J_0^{-1} \{ T(q^{(\ell)}(\eta)) - x_0 - h\eta \} \right]_{\leq m}. \quad (\text{A.16})$$

The brackets mean truncation by total degree. To see why this works, suppose the current inverse-series error begins at degree r . Near the base point, the linear part of the composition error is J_0 times that error. Multiplication by J_0^{-1} cancels its degree- r term; variation of the Jacobian contributes only higher-degree terms. Each correction thus fixes at least one additional degree. Three corrections recover degrees one through three in exact arithmetic; the prototype uses four to include a redundant residual correction. This is a local series construction, not a global guarantee of convergence for arbitrary physical displacements.

The geometry fixtures are explicit polynomial maps, so their composition with a jet is available directly. The reference finite-element polynomial is expressed in centred monomials by a basis conversion, then evaluated at $q(\eta)$. That conversion preserves the represented function in exact arithmetic; it is not a physical-polynomial reconstruction. Its conditioning, the inverse-series arithmetic and the factor $h^{-|\alpha|}$ can nevertheless amplify floating-point errors.

A.1.6 What direct differentiation improves, and what it cannot fix

Let $\mathcal{R}_\alpha u_h$ denote the production recursively projected derivative in its chosen order, and let u be an underlying smooth exact field. Then

$$\mathcal{R}_\alpha u_h - \partial^\alpha u = \underbrace{\mathcal{R}_\alpha u_h - \partial^\alpha u_h}_{\text{derivative-operator discrepancy}} + \underbrace{\partial^\alpha (u_h - u)}_{\text{differentiated input error}}, \quad (\text{A.17})$$

$$\partial^\alpha u_h - \partial^\alpha u = \partial^\alpha (u_h - u). \quad (\text{A.18})$$

Direct differentiation removes the intermediate-projection contribution. It does not remove error already present in u_h , and differentiation can magnify that error. The two terms in the first line may reinforce or cancel each other. Removing the first term therefore need not decrease the total error against u , even though it gives a more faithful derivative of u_h . Intermediate projection can also suppress some components of the input error. These identities explain why an exactly represented test field is useful for isolating operator error, and why it cannot establish improved PDE performance on its own.

For sufficiently smooth full-dimensional maps and inputs, exact mixed Cartesian derivatives commute. The xyx , xyx and yxx derivatives of u_h agree. The jet represents them by the single coefficient with $\alpha = (2,1)$; it does not independently calculate three differentiation paths. By contrast, successive finite-space projections can make those paths differ, as derived in Section 3.4.5. Agreement built into a representation is not an independent correctness check, which is why the isolated experiment also used physical-coordinate finite differences with an independent inverse-map calculation.

In principle, derivatives required only at face quadrature points could be evaluated directly there and their one-sided jumps formed afterwards. The OFDG damping still uses its physical nested projections; changing the sensor derivative evaluator would not remove those projections. Such an integration would require geometry derivatives or a suitable differentiable map evaluator, pointwise derivative evaluation and verification of the resulting damping rates. The experiment supports two explicit polynomial geometry families in two dimensions, through third derivatives. It does not provide this interface for arbitrary MFEM maps, NURBS or production 3D filtering, and it has not replaced the current method.

A.2 The mathematics needed for the first-order surface example

A.2.1 Two surface coordinates and three physical components

A surface patch $K \subset \Gamma \subset \mathbb{R}^3$ is parameterized by $X(\xi_1, \xi_2)$. Its Jacobian has two columns and three rows:

$$J = (e_1 \ e_2), \quad e_a = \partial_{\xi_a} X \in \mathbb{R}^3, \quad G = J^T J. \quad (\text{A.19})$$

The columns e_1, e_2 span the tangent plane. They need not have length one or be perpendicular. The 2×2 matrix G , called the metric matrix, records their lengths and mutual angle: $G_{ab} = e_a \cdot e_b$. For a regular surface patch these vectors are independent, so G is positive definite. Although the surface lies in three-dimensional space, it has only two independent directions of motion. There is no square inverse J^{-1} .

A surface DG function is represented as $u_h(X(\xi)) = \hat{u}_h(\xi)$. Surface values do not specify how a function changes when leaving the surface in the normal direction. This is why an ordinary ambient x, y or z derivative is not determined by the surface function alone. The tangential gradient is determined: it measures changes along admissible surface directions.

A.2.2 Deriving the tangential gradient

Seek a tangent vector g whose dot product with a tangent displacement reproduces the directional change of u . A reference displacement $\delta\xi$ induces $\delta x = J\delta\xi$. Therefore

$$g \cdot J\delta\xi = (\nabla_{\xi}\hat{u}) \cdot \delta\xi \quad \text{for every } \delta\xi. \quad (\text{A.20})$$

This means $J^T g = \nabla_{\xi}\hat{u}$. Because g is tangent, write $g = Jc$ for a two-component coefficient vector c . The resulting system is $Gc = \nabla_{\xi}\hat{u}$, giving

$$\boxed{\nabla_{\Gamma} u = JG^{-1}\nabla_{\xi}\hat{u}.} \quad (\text{A.21})$$

The result has three Cartesian components, but lies in a two-dimensional tangent plane. Keeping three components avoids constructing and comparing a separate orthonormal tangent basis on every element.

An equivalent physical description uses a unit normal n and the matrix

$$P = I - nn^T = JG^{-1}J^T. \quad (\text{A.22})$$

For a vector a , $Pa = a - n(n \cdot a)$ removes its normal component. If U is any smooth ambient extension of u , then

$$\nabla_{\Gamma} u = P\nabla U. \quad (\text{A.23})$$

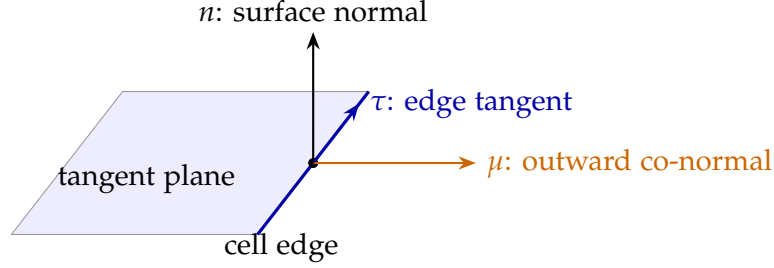


Figure A.1: Local schematic at a surface-cell edge, drawn in perspective. The edge tangent τ and outward co-normal μ lie in the tangent plane and are perpendicular in physical space. The surface normal n points out of that plane. Transport across the edge uses $v \cdot \mu$, not $v \cdot n$.

Different extensions can have different normal derivatives. Their tangential derivatives agree, because they have identical values along every surface curve. Thus the right-hand side does not depend on an arbitrary choice of extension. The matrix P is an exact geometric projection of a vector at one point; it is not the finite-element L^2 projection Π_K .

A.2.3 Surface area, edge length and the co-normal

A small reference parallelogram has physical area $|e_1 \times e_2|$ times its reference area. Consequently

$$dS = \sqrt{\det G} d\zeta_1 d\zeta_2, \quad |K| = \int_{\hat{K}} \sqrt{\det G} d\zeta. \quad (\text{A.24})$$

A reference edge parameterized by $\hat{\gamma}(s)$ has physical tangent $J\hat{\gamma}'(s)$ and line element

$$d\ell = |J\hat{\gamma}'(s)| ds. \quad (\text{A.25})$$

A flux leaves a surface cell through its edge. Its relevant outward direction must be tangent to the surface and perpendicular to the edge. This unit vector is the *co-normal* μ , distinct from the surface normal n . If $\hat{v}$ is the outward unit reference-edge normal, then

$$\mu = \frac{JG^{-1}\hat{v}}{|JG^{-1}\hat{v}|}, \quad \mu d\ell = \sqrt{\det G} JG^{-1}\hat{v} d\hat{\ell}. \quad (\text{A.26})$$

Indeed, its dot product with $J\hat{\tau}$ is zero whenever $\hat{v} \cdot \hat{\tau} = 0$, so it is perpendicular to the edge within the tangent plane. The second identity combines this direction with the physical edge scale. It is useful for conservative flux assembly. Figure A.1 distinguishes these three directions. On the smooth sphere, neighbouring cells have opposite co-normals at their common edge. For a merely piecewise-planar approximation of a general surface, this statement requires additional care because the two tangent planes can differ; the prototype uses an exact smooth radial sphere map.

A.2.4 The exact sphere map used in the prototype

Let $F_h(\xi)$ denote the base element map from an octahedral or cubed-sphere mesh, with $F_h(\xi) \neq 0$. The actual geometry used at quadrature points is

$$X(\xi) = \frac{F_h(\xi)}{|F_h(\xi)|}, \quad J = \frac{1}{|F_h|} (I - XX^\top) DF_h. \quad (\text{A.27})$$

The second formula follows by differentiating the normalization. Every mapped point has unit length. The tangent projector is $I - XX^\top$, since X itself is the outward unit normal on the unit sphere. The base mesh supplies connectivity and reference coordinates; assembly, projection, error measurement and visualisation use the radial map. This is not the same as solving on planar facets whose vertices happen to lie on the sphere. It also is not a general change to MFEM's geometry API.

The metric, gradient and co-normal constructions above apply to any smooth regular embedded surface patch. The particular radial map and rotation problem are sphere-specific. General mathematical formulas do not imply that the prototype implements all such surfaces or their interfaces.

A.2.5 Surface transport and the conservative DG form

For a tangential velocity v , the conservative transport equation is

$$\partial_t u + \text{div}_\Gamma(vu) = 0. \quad (\text{A.28})$$

Surface divergence measures net tangential flux leaving a small surface region per unit area. For a test function w_h , integration by parts on a cell gives

$$\int_K w_h \partial_t u_h dS - \int_K \nabla_\Gamma w_h \cdot v u_h dS + \int_{\partial K} w_h \hat{f} d\ell = 0. \quad (\text{A.29})$$

On an edge oriented outward from K^- , write $b = v \cdot \mu^-$ and let u^-, u^+ be the two traces. The upwind flux is

$$\hat{f} = b \begin{cases} u^-, & b \geq 0, \\ u^+, & b < 0. \end{cases} \quad (\text{A.30})$$

The same numerical flux enters the neighbouring equation with the opposite sign. Choosing $w_h = 1$ cancels interior fluxes when summing over cells, which explains global mass conservation on the closed sphere. The normal speed $v \cdot n$ cannot replace b : tangential motion has $v \cdot n = 0$ even while it transports material across cell edges.

The example uses $v = (-y, x, 0)$, rotation about the z axis. This velocity is tangent and surface-divergence-free. Tracing a characteristic backwards gives

$$u(x, y, z, t) = u_0(x \cos t + y \sin t, -x \sin t + y \cos t, z). \quad (\text{A.31})$$

At $T = \pi/2$ the pattern has completed a quarter rotation. This provides an exact reference without solving a second numerical PDE. The surface is closed, so there is no physical boundary condition to impose.

A.2.6 Projected first derivatives and the order-one damping

The isolated solution space is discontinuous P1 on triangles or Q1 on quadrilaterals. Let $\varphi_b = \hat{\varphi}_b \circ X^{-1}$ on a patch, where X^{-1} means its local inverse from the surface patch to its reference coordinates. For Cartesian component $a \in \{1, 2, 3\}$, define

$$g_{K,a} = \Pi_K[(\nabla_\Gamma u_h)_a]. \quad (\text{A.32})$$

Here Π_K is the physical surface L^2 projection into that same local scalar DG space. Testing with each basis function gives

$$M^K Z_a = B_a^K U, \quad (\text{A.33})$$

$$M_{ij}^K = \int_{\hat{K}} \hat{\varphi}_i \hat{\varphi}_j \sqrt{\det G} d\hat{\xi}, \quad (\text{A.34})$$

$$(B_a^K)_{ij} = \int_{\hat{K}} \hat{\varphi}_i (JG^{-1} \nabla_{\hat{\xi}} \hat{\varphi}_j)_a \sqrt{\det G} d\hat{\xi}. \quad (\text{A.35})$$

The vector Z_a contains coefficients of the projected scalar component. Although the exact gradient is tangent at every point, separately projecting its three components need not preserve pointwise tangency: the tangent plane varies across the element. The prototype uses these three projected components as its sensor variables. It does not project the resulting vector tangentially a second time.

For an edge F , brackets denote the difference of the two one-sided traces at the same physical point. The two edge-averaged jump measures are

$$J_{F,0} = \frac{1}{|F|} \int_F |[u_h]| d\ell, \quad J_{F,1} = \frac{1}{|F|} \int_F \sum_{a=1}^3 |[g_a]| d\ell. \quad (\text{A.36})$$

The second expression retains a componentwise absolute sum in fixed ambient Cartesian axes. It is not a rotationally invariant vector norm, and no such invariance is claimed. Changing reference coordinates consistently does not change the exact geometric gradient, but changing the ambient Cartesian axes can change this particular sensor sum.

Let $h_K = \sqrt{|K|/|\hat{K}|}$, let β_F be the sampled maximum of $|v \cdot \mu|$ on the edge, and let A be the sampled global maximum of $|u_h - \bar{u}_\Gamma|$, where $\bar{u}_\Gamma$ is the global surface mean. Setting $k = 1$ in Eq. (3.23) gives the prefactors $(2l + 1)/(2l!)$: $1/2$ for $l = 0$ and $3/2$ for $l = 1$. The only nonconstant shell receives both rates. With the surface jump measures above, the tested rate

and update are

$$\lambda_K = \frac{1}{A} \sum_{F \in \partial K} \beta_F \left(\frac{J_{F,0}}{2h_K} + \frac{3}{2} J_{F,1} \right), \quad (\text{A.37})$$

$$u_h^{\text{new}} = \bar{u}_K + \exp(-\Delta t \lambda_K)(u_h^{\text{old}} - \bar{u}_K), \quad \bar{u}_K = \frac{1}{|K|} \int_K u_h dS. \quad (\text{A.38})$$

Both terms in the rate have units of inverse time after division by A : one jump is divided by a length, and the derivative jump already has one inverse length. The near-constant guard sets damping inactive when $A \leq 10^{-12} \max(1, \text{sampled volume magnitude})$, avoiding division by a vanishing amplitude. The factor is applied once after a complete RK4 step, with the rate evaluated from that step's input to the filter.

The cell mean is preserved because the damped difference integrates to zero. On Q1, all nonconstant modes, including the bilinear reference mode, belong to this damped tail. Only first derivatives enter this construction; there is no repeated tangential derivative. Positive surface quadrature defines the discrete integrals consistently. No positivity limiter or KXRCF is included, and the mean-preserving update alone does not prove positivity.

A.3 Why higher surface derivatives are a separate extension

SCOPE LIMITATION

This section is explanatory mathematics only. Second- and higher-order surface derivative sensors have not been implemented or adopted. It records why increasing the polynomial order would require a new decision, rather than merely enabling the volume derivative recursion.

A.3.1 Ambient and surface second derivatives answer different questions

Let $U(x, y, z) = x$ in ambient space and restrict it to the unit sphere: $u = U|_\Gamma$. Certainly $U_x = 1$ and $U_{xx} = 0$. Those derivatives vary x while holding y, z fixed, which generally leaves the sphere. Along the unit-speed great circle

$$\gamma(s) = (\cos s, \sin s, 0), \quad u(\gamma(s)) = \cos s, \quad (\text{A.39})$$

the first and second derivatives with respect to distance s are $-\sin s$ and $-\cos s$. There is no contradiction. For any smooth ambient function and path, the ordinary chain rule gives

$$\frac{d^2}{ds^2} U(\gamma(s)) = \gamma'(s)^\top (H_U) \gamma'(s) + \nabla U \cdot \gamma''(s). \quad (\text{A.40})$$

The first term measures change of the gradient; the second accounts for change in the direction of travel. For $U = x$, the first term is zero and $\nabla U = (1, 0, 0)$, while $\gamma'' = (-\cos s, -\sin s, 0)$. The second term therefore equals $-\cos s$. A straight line and a curved path through the same ambient field need not produce the same second derivative along distance.

A.3.2 A physical description of the surface Hessian

The intrinsic surface Hessian describes how the tangential gradient changes along the surface, with its output compared in the tangent plane. If $g = \nabla_\Gamma u$, its ambient matrix representation is

$$H_\Gamma u = P Dg P. \quad (\text{A.41})$$

Here Dg has entries $(Dg)_{ij} = \partial_{x_j} g_i$ for a smooth local extension. The right P restricts the differentiation direction to tangent vectors. The left P keeps only the tangent part of the resulting change. The value on tangent directions is independent of how g was extended away from the surface. These geometric projections do not introduce an L^2 approximation.

Since $g = P\nabla U$, differentiating g also differentiates P . Surface curvature appears precisely because the tangent plane changes with position. Using only $PH_\Gamma P$ would omit that effect. On the unit sphere, write $p = (x, y, z)$, so $P = I - pp^\top$. For $u = x$,

$$g = e_x - xp, \quad H_\Gamma u = -xP. \quad (\text{A.42})$$

To verify the latter, take a tangent vector t . Differentiating g in that direction gives $-t_x p - xt$; tangential projection removes the first term and leaves $-xt$. Thus for a unit tangent t , $t^\top H_\Gamma u t = -x$. Along the great circle this is $-\cos s$, exactly as above. In particular, it is a surface derivative, not the ambient Cartesian quantity U_{xx} .

A general surface curve also has a tangential acceleration. Its second derivative of u contains the corresponding first-gradient contribution. For a geodesic, meaning a surface path with zero tangential acceleration, that contribution vanishes, so the surface Hessian along its tangent gives the second derivative along the path. Great circles parameterized by arc length are such paths on the sphere. This qualification matters: not every curve on a surface measures the Hessian alone.

A.3.3 The same construction in reference coordinates

The reference-coordinate description expresses the same geometric quantity in the moving basis $e_a = \partial_a X$. It is not another physical derivative notion.

A.3. Why higher surface derivatives are a separate extension

Write $G_{ab} = e_a \cdot e_b$ and let G^{ab} denote entries of G^{-1} , not componentwise reciprocals. Define the coefficients

$$C_{ab}^c = \sum_{d=1}^2 G^{cd} e_d \cdot \partial_a \partial_b X. \quad (\text{A.43})$$

These are commonly called Christoffel symbols. Their concrete meaning is that the tangential part of the changing basis vector $\partial_a e_b$ is $\sum_c C_{ab}^c e_c$. The remaining part points normally to the surface.

The scalar first derivative is $\partial_b \hat{u} = g \cdot e_b$. Differentiating this identity with respect to ξ_a gives

$$\partial_a \partial_b \hat{u} = (\partial_a g) \cdot e_b + g \cdot \partial_a e_b. \quad (\text{A.44})$$

The second term measures change of the basis vector itself. Subtracting its tangential contribution isolates change of the gradient:

$$H_{ab} = \partial_a \partial_b \hat{u} - \sum_{c=1}^2 C_{ab}^c \partial_c \hat{u}. \quad (\text{A.45})$$

These are the surface Hessian evaluated on the coordinate tangent vectors. Recovering its ambient matrix representation uses $H_{\Gamma} u = J G^{-1} H G^{-1} J^T$. Hence the coordinate formula and Eq. (A.41) describe the same object. The correction is required even though the reference second derivatives of the scalar function can be evaluated easily.

A.3.4 Why third order is not merely another scalar multi-index

Differentiating a scalar once gives a gradient; differentiating that gradient intrinsically gives a two-slot object, the Hessian. A further derivative must account for changes in both of its tangent arguments. With the first index specifying the differentiation direction, its coordinate components are

$$T_{abc} = \partial_a H_{bc} - \sum_{d=1}^2 C_{ab}^d H_{dc} - \sum_{d=1}^2 C_{ac}^d H_{bd}. \quad (\text{A.46})$$

Each subtraction corrects for one moving tangent-basis vector. In general, a tensor with r such arguments acquires r corresponding correction terms on differentiation. Direct third-order evaluation generally requires geometry derivatives through third order. Repeatedly differentiating three Cartesian components as if they were unrelated scalar fields does not, by itself, specify these intrinsic tensor derivatives.

There is also a fundamental difference from Cartesian volume derivatives. The surface Hessian of a smooth scalar field is symmetric, but its covariant third derivative need not be fully symmetric on a curved surface. On the unit

sphere the preceding example gives $H_{bc} = -xG_{bc}$. The intrinsic derivative preserves the metric, so

$$T_{abc} = -(\partial_a x)G_{bc}. \quad (\text{A.47})$$

At the north pole choose orthonormal tangent directions $e_1 = e_x$ and $e_2 = e_y$. Then $T_{122} = -1$ whereas $T_{212} = 0$. This difference exists in exact geometry and exact arithmetic. It is not the projection-order error measured for volume derivatives. Symmetrizing all slots would define a different sensor quantity, rather than correcting that difference as a numerical defect.

Bibliography

- [1] Jan S. Hesthaven and Tim Warburton. *Nodal Discontinuous Galerkin Methods: Algorithms, Analysis, and Applications*, volume 54 of *Texts in Applied Mathematics*. Springer, 2008.
- [2] Yong Liu, Jianfang Lu, and Chi-Wang Shu. An essentially oscillation-free discontinuous Galerkin method for hyperbolic systems. *SIAM Journal on Scientific Computing*, 44(1):A230–A259, 2022.
- [3] Jianfang Lu, Yong Liu, and Chi-Wang Shu. An Oscillation-free Discontinuous Galerkin Method for Scalar Hyperbolic Conservation Laws. *SIAM Journal on Numerical Analysis*, 59(3):1299–1324, January 2021.
- [4] Manting Peng, Zheng Sun, and Kailiang Wu. OEDG: Oscillation-eliminating discontinuous Galerkin method for hyperbolic conservation laws. *Mathematics of Computation*, 94(353):1147–1198, 2025. Published online in 2024.



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

- ☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies¹.
- ☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Authored by:

If the work was compiled in a group, the names of all authors are required.

Last name(s):

First name(s):

With my signature I confirm the following:

- I have adhered to the rules set out in the [Citation Guidelines](#).
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date

Signature(s)

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

¹ For further information please consult the ETH Zurich websites, e.g. <https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html> and <https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html> (subject to change).